KubeSight

Security audit

Read-only security review of the whole KubeSight codebase on master, carried out by eight audit agents working in parallel.

1	19	31	38	6
CRITICAL	HIGH	MEDIUM	LOW	INFO

Repository: kubesigh_1 · branch master · commit dad6d8b6 (2026-10-11)
Covers everything on master, including the 68 commits since the 2026-09-26 re-audit
Report date: 2026-10-12 · Confidential: contains vulnerability details. All secret values are masked.

Contents

1. Executive summary 3

2. What to fix first 4

3. Scope and method 6

4. Status of earlier findings 8

5. All findings at a glance 9

6. Detailed findings 14

6.1 Authentication, sessions and RBAC . . . 14

6.2 Kubernetes cluster access and mutation . . . 20

6.3 Native CI platform . . . 24

6.4 Integrations, webhooks and outbound requests . . . 30

6.5 MCP server and Hermes agent . . . 35

6.6 Cluster Builder, SSH and VM provisioning . . . 40

6.7 Approvals, promotions and governed delivery . . . 42

6.8 Frontend, infrastructure, secrets and dependencies . . . 46

7. Dependency scan 53

8. What is already done well 54

9. Remediation roadmap 58

1. Executive summary

KubeSight was audited end to end across eight areas: authentication and RBAC, Kubernetes access, the native CI platform, integrations, the MCP server and Hermes, Cluster Builder and VM provisioning, approvals, and the platform (frontend, infrastructure, secrets, dependencies). After duplicates across areas were merged, there are **95 distinct findings: 1 Critical, 19 High, 31 Medium, 38 Low and 6 Info**.

The findings fall into six themes.

1. Credentials that work without any bug.

- The default accounts `admin/admin123`, `operator/operator123` and `viewer/viewer123` log in without MFA, and they are re-created on every boot, even after they are deleted (AUTH-01, the only Critical).
- Two KubeSight API tokens and a Telegram bot token are in git history that is pushed to GitHub and Bitbucket (MCP-01, PLAT-02).
- A local branch holds LLM-provider refresh tokens and another bot token (PLAT-06).

These need no exploit, only access to the login page or the repository, so they come first.

2. Permission boundaries that do not hold.

- The built-in viewer role can stage arbitrary manifests in a change bundle. On a cluster that needs 0 approvals, KubeSight then applies them with its own credentials, and any bundle reader can see Secret values (K8S-01, K8S-02).
- Adding a second cluster parameter to a request gets past the cluster-access check (AUTH-04).
- `users:update`, `users:create` and `roles:manage` all lead to Admin (AUTH-02, AUTH-03, the earlier H1).
- `clusters:update` can point a cluster's stored credentials at another endpoint (K8S-03).
- The email approval result page has a stored XSS that a viewer can plant (APR-02).

3. Gaps in the approval gate.

- Pod exec, from the UI or MCP, skips the cluster approval gate (K8S-05).
- Approvers are not limited to the configured pool or to their own clusters, and the quorum is silently lowered (APR-06).
- Email links still approve on GET (APR-04).
- An approved deployment window has no maximum length (APR-07).

4. CI supply chain. Builds share one writable cache, so one service's build can run code in another's (CI-01). Any earlier stage can run code inside the image stage that holds the registry push credential (CI-02). The Deploy stage's authorization covers the target, not the code that is deployed (CI-03), and a CI admin can mount any node path into build pods (CI-04).

5. The AI agent boundary. Hermes ticket tools are not tied to the ticket being handled, so text in one ticket can make Hermes read or act on others (MCP-03). The rule that Hermes may only use ticket tools exists only in its prompt (MCP-04). Any ticket author can trigger changes that run without a human in the loop (MCP-05). The MCP per-cluster Off and Read only modes can be bypassed through CI tools (MCP-06).

6. Platform. The `nodes/proxy` role that the repo recommends for every cluster also allows exec into any pod (PLAT-05). The backend runs as root (PLAT-12) on the external ingress class (PLAT-13), and the dependency caps block known fixes (PLAT-08).

What has improved since 2026-09-26.

- Kubeconfigs are encrypted at rest.
- Chart archive symlinks are refused.
- The CI build-script injection (H4) is fixed.
- Ticketing webhooks fail closed.
- YAML, Helm, restart, scale and rollback share one approval gate.
- The requester cannot approve their own bundle.

The new OpenTofu provisioning code was built carefully: no credentials in the config, a locked-down state backend, and the approved plan is the one that is applied. No finding above Medium was found in that area.

Findings by area and severity

Area	Critical	High	Medium	Low	Info	Total
Authentication, sessions and RBAC	1	3	4	8	2	18
Kubernetes cluster access and mutation	–	5	2	2	–	9
Native CI platform	–	4	4	6	–	14
Integrations, webhooks and outbound requests	–	1	5	6	1	13
MCP server and Hermes agent	–	3	3	4	–	10
Cluster Builder, SSH and VM provisioning	–	–	2	3	–	5
Approvals, promotions and governed delivery	–	1	5	5	–	11
Frontend, infrastructure, secrets and dependencies	–	2	6	4	3	15
All areas	**1**	**19**	**31**	**38**	**6**	**95**

2. What to fix first

Today: operational, no code change.

1. On every installation, disable the `admin`, `operator` and `viewer` accounts or change their passwords. Do not delete them: deleted accounts come back on the next boot with the known password (AUTH-01).
2. Revoke these credentials, then purge both files from history on every branch and force-push both remotes:
   - the `ksa_4a28***` token in connect-hermes-to-kubesight.sh (MCP-01);
   - the `ksa_` token and the Telegram bot token in `k8s/hermes-secret.yaml` (PLAT-02).
3. Delete the `backup-before-clean-push` branch and run `git gc`. Revoke the OpenAI-Codex and Nous sessions and rotate the bot and `ksa_` tokens it contains (PLAT-06).
4. Take `change_bundles:create` and `change_bundles:view` away from the viewer role, and review operator and cluster_admin (K8S-01, K8S-02). Until K8S-01 is fixed in code, no cluster or global default should require 0 approvals.
5. Find every cluster where `k8s/node-stats-rbac.yaml` was applied. Treat that role as equivalent to cluster-admin and remove it where it is not needed (PLAT-05).
6. Check that the token Hermes uses for tickets belongs to the `hermes-agent` account, not an operator or admin (MCP-04).

This week: small, contained code changes.

7. Escape the values on the email approval result pages (APR-02), a one-line `html.escape`.
8. In `require_cluster_access`, resolve the cluster once and have handlers read that value; reject requests with conflicting values (AUTH-04).
9. Let a user grant only roles and permissions they hold themselves, and let only admins modify admins (AUTH-02, AUTH-03).
10. Put pod exec behind the approval gate and a new `pods:exec` permission (K8S-05).
11. Make the seed create a single bootstrap admin, only when the users table is empty, with a forced password change (AUTH-01).
12. Set `MAX_CONTENT_LENGTH` and lower the ingress body size outside the upload routes (AUTH-08, CI-06).
13. Lift the dependency caps: gunicorn ≥ 23, flask-cors ≥ 6, cryptography ≥ 46.0.5 (PLAT-08).

Next: design work, in this order.

14. **Change bundles:** make staging require deploy rights and access to the cluster and namespace, and redact bundle payloads (K8S-01, K8S-02).
15. **CI isolation:** give each service its own cache subPath, stop sourcing build.env in the image stage, stamp the Deploy stage over the whole pipeline, and restrict cache volume backings (CI-01 to CI-04).
16. **Hermes:** bind ticket tools to the current ticket, enforce a server-side tool allowlist for `hermes-agent`, and add a requester allowlist (MCP-03 to MCP-06).
17. **Application Intelligence:** isolate the build verifier from the credentialed containers (INT-01).

18. **Cluster credentials and YAML:** require credentials again when a cluster endpoint changes, and replace the kind denylist with an allowlist for non-admins (K8S-03, K8S-04).

3. Scope and method

- **Target.** master at commit dad6d8b6 (2026-10-11), with particular attention to the 68 commits since the 2026-09-26 re-audit: the approvals hub, Hermes VM requests, OpenTofu provisioning, CI webhook triggers, the Deploy stage, shared pipelines, promotion timetables and the MCP access controls.
- **Agents.** Eight audit agents ran in parallel, each on one area:
  - authentication and RBAC, including a sweep of all 645 registered routes;
  - Kubernetes cluster access;
  - native CI;
  - integrations and outbound requests;
  - the MCP server (all 105 tools) and Hermes;
  - Cluster Builder, SSH and OpenTofu;
  - approvals and governed delivery;
  - frontend, infrastructure, secrets and dependencies.
- **Read-only.** No repository file was changed. No real database, kubeconfig, cluster or external service was touched, and Docker was not used. Some findings were reproduced against an in-memory SQLite copy of the app in mock mode; these are marked "Confirmed (PoC)".
- **Evidence standard.** Every finding was traced from the route, through the permission check, to the service that acts. "Confirmed" means the code path was read end to end or reproduced; "Likely" means the code path is clear but the impact depends on runtime configuration that is not in the repository.
- **Spot checks.** APR-02 (the escaping gap), AUTH-04 (the decorator's lookup order) and PLAT-02 (the commit is an ancestor of origin/master) were re-verified directly during report assembly.
- **De-duplication.** Several agents reached the same root cause from different sides. Sixteen duplicates were merged into one finding each, and IDs were kept stable, so gaps in a numbering sequence are merged items:

Merged ID	Now reported as
APR-01, APR-03, APR-09, APR-15	K8S-01, K8S-02, K8S-06, K8S-11
K8S-07, K8S-09	APR-04, APR-06
MCP-02	K8S-05
MCP-07	PLAT-06
AUTH-17	PLAT-11
PLAT-01, PLAT-03, PLAT-04	AUTH-01, MCP-01, CI-01
PLAT-07, PLAT-09, PLAT-10, PLAT-15	AUTH-08, AUTH-09, CI-05, AUTH-11

- **Limits.**
  - Nothing was tested against a live cluster.
  - Whether the leaked tokens are still active could not be checked.
  - pip-audit was not available, so Python advisories come from published CVE data as of mid-2026. The frontend audit used `npm audit` on the lockfile.
  - The backend Deployment and its RBAC manifests are not in the repository, so the backend's own pod security settings could not be reviewed.

Severity scale

Severity	Meaning in this report
CRITICAL	Unauthenticated or low-privilege path to cluster or host compromise, secret theft, or authentication bypass.
HIGH	Authenticated privilege escalation, remote code execution, action that bypasses an approval or cluster scope, or secrets exposed to unprivileged users.
MEDIUM	Real attack path that needs specific conditions or privileges, or has limited impact.
LOW	Hardening gap with little direct impact.

INFO	Observation; no direct risk.

4. Status of earlier findings

This table shows the status of every item still open or newly fixed since the 2026-09-26 re-audit (KubeSight-Audit-2026-09-26-production-hardening.pdf), with the finding in this report that now tracks it.

Earlier item	Status on 2026-10-11	Tracked as
H1: users:update can make yourself Admin	**Still open**, and wider: users:create and roles:manage also escalate	AUTH-02, AUTH-03
Seeded default users (owner-accepted)	**Still open**, and worse: deleted accounts are re-created on boot	AUTH-01
Committed ksa_ token (owner-accepted)	**Still open**; a second token is also in history	MCP-01, PLAT-02
pods:exec	**Still open**; exec also skips the approval gate	K8S-05
GET-based email approvals	**Still open** (bundles and deployment requests)	APR-04
Empty-email quorum bypass	**Still open in code**; harder to reach, since new users need an email	K8S-06
SSRF	**Still open** in Helm repos, cluster hosts, integrations, health checks and post-action webhooks	K8S-08, INT-02, INT-04, CI-10
Missing MAX_CONTENT_LENGTH	**Still open**	AUTH-08, CI-06
CORS *	**Still open**	AUTH-11
flask-cors / gunicorn CVE ranges	**Still open**; cryptography is also capped	PLAT-08
Onboarding token reuse	**Still open**	AUTH-07
Containers run as root	**Still open**	PLAT-12
Helm values exposure	Partly fixed: masking guesses from key names	–
H4: CI build-script injection	Fixed	–
H2: Helm read ACL; H3: kubectl flag injection	Fixed (names validated, argv screened)	–
Plaintext kubeconfigs	Fixed (Fernet at rest, 0600 per-call copies)	–
Chart archive symlink read	Fixed	–
Approval gate only on apply_yaml	Fixed for YAML, Helm, restart, scale and rollback	–
Bundle self-approval	Fixed	–
Ticketing webhooks without a secret	Fixed (fail closed, constant-time compare)	–
Werkzeug debugger in production; /health counts; backend/data in the image	Fixed	–

5. All findings at a glance

ID	Severity	Finding	Who can exploit
AUTH-01	CRITICAL	Default accounts with public passwords and no MFA are re-created on every boot	Unauthenticated: anyone who can reach the login page.
APR-02	HIGH	The email approval result page renders the cluster name unescaped (stored XSS)	`deployment_requests:request`, which viewer, operator and cluster_admin have by default.
AUTH-02	HIGH	users:update and users:create are equivalent to Admin (H1, wider than reported)	A non-admin whose role holds `users:update`, `users:create` or `users:manage`.
AUTH-03	HIGH	roles:manage can give its own role every permission	Holders of `roles:manage`, or of `users:manage`, which implies it.
AUTH-04	HIGH	The cluster-access check can be bypassed by sending a second cluster parameter	A logged-in user scoped to at least one cluster who also has the route's permission.
CI-01	HIGH	One shared cache volume, mounted whole into every build, lets any build poison other services' builds	Anyone whose code runs in any build: users with `ci_pipelines:edit`, users with `ci_builds:run` building a branch they control, and external people who can push a branch …
CI-02	HIGH	Any earlier stage can run code inside the image-build stage, which holds the registry push credentials	Any code that runs before a `container_image` stage.
CI-03	HIGH	The Deploy stage's authorization stamp covers the target, not the code that gets deployed	Users with `ci_pipelines:edit` and/or `ci_services:edit` who lack `apps:deploy` or access to the namespace.
CI-04	HIGH	`ci_runners:manage` can mount any node path or NFS export into untrusted build pods	Users with `ci_runners:manage`, a CI-admin permission rather than KubeSight Admin.
INT-01	HIGH	Build-verifier code shares a writable HOME and workspace with the credentialed containers in Application Intelligence jobs	An author of code in an analyzed repository, when that code is analyzed in Build Verified mode or turned into a pull request.
K8S-01	HIGH	Viewer-level users can stage arbitrary manifests that the executor applies with KubeSight's credentials, with kind restrictions lifted	Any logged-in user with `change_bundles:create`, which the built-in viewer, operator and cluster_admin roles have.
K8S-02	HIGH	Staged Secret values and Helm values are stored and returned unredacted to any bundle reader	Anyone with `change_bundles:view` (default viewer role), for bundles on any cluster, whatever their cluster or namespace grants.
K8S-03	HIGH	`clusters:update` can repoint any cluster's stored credentials to another API endpoint; management routes are not cluster-scoped	Users with `clusters:update` (plus `clusters:test` to start a connection).
K8S-04	HIGH	YAML/image/wizard apply relies on a kind denylist; namespace-scoped deployers can exceed their namespace grant	`apps:deploy` holders limited to certain namespaces.
K8S-05	HIGH	Pod exec (UI and the kubesight_pod_exec MCP tool) bypasses the cluster approval gate and has no dedicated permission (prior pods:exec)	`apps:deploy` holders with access to the namespace, on any cluster including approval-gated ones, and any MCP token with that permission (including Hermes if its token ha…
MCP-01	HIGH	A live-format KubeSight API token is committed and pushed to both remotes (previously owner-accepted)	Anyone with read access to the GitHub or Bitbucket repository, or any clone of it.
MCP-03	HIGH	Ticket tools are not bound to the ticket Hermes was given, so a prompt-injected ticket can read and act on every other ticket	Anyone who can create a ticket, add a comment, or reply by email in the connected Zoho Desk department or Jira project (often external requesters).

ID	Severity	Finding	Who can exploit
MCP-04	HIGH	The ticket agent's "use only ticket tools" rule exists only in the prompt, and the design runs it on the operators' shared Hermes and token	A ticket author (as in MCP-03), whenever the Hermes at `TICKET_AGENT_HERMES_URL` holds a token broader than the `hermes-agent` role.
PLAT-02	HIGH	A live-format Telegram bot token and a second ksa_ token are in master history, pushed to GitHub and Bitbucket	Anyone with read access to the GitHub repository (12048785 is an ancestor of origin/master) or to the Bitbucket repository (it is in main / bitbucket-main), and anyone ho…
PLAT-05	HIGH	The nodes/proxy get ClusterRole allows exec in every pod on every node, contrary to the manifest's comment	Anyone who obtains the backend identity: a backend RCE (the backend runs as root, PLAT-12), a leaked service-account token, or the static token copy in /root/.kube/config.
APR-04	MEDIUM	Email approval links change state on GET, are reusable, and are bound to a mailbox rather than an account	Anyone who receives the link: shared or distribution-list mailboxes, forwarded mail, URL-scanning gateways.
APR-05	MEDIUM	Approval rule changes, individual votes and some status changes are not audited	`deployment_requests:manage` (admin by default) for rule changes;
APR-06	MEDIUM	Who may approve is not tied to the approver pool or to cluster access, and the quorum is silently reduced	`change_bundles:manage` or `deployment_requests:manage` (admin by default;
APR-07	MEDIUM	A deployment request approval has no time limit and no content binding	`deployment_requests:request`, once one approval is obtained.
APR-08	MEDIUM	CI Deploy stages render inventory templates at deploy time, and any template editor can change them	`inventory:update` (cluster_admin by default).
AUTH-05	MEDIUM	Anyone can lock any account, up to an admin-only unlock	Unauthenticated;
AUTH-06	MEDIUM	API tokens keep working on locked accounts	Anyone holding a `ksa_` token of a user that an admin locked (for example a compromised account) or that was auto-locked.
AUTH-07	MEDIUM	The onboarding token still works after onboarding and can change the password	Anyone who obtains a user's onboarding token within its 30-minute lifetime.
AUTH-08	MEDIUM	No request size limit, including on unauthenticated routes	Unauthenticated, via /api/auth/login, inbound webhooks, CI hooks and similar routes.
CI-05	MEDIUM	Shared BuildKit daemon runs without a process sandbox, unconfined, with no client authentication	Anyone who controls a built Dockerfile (repo committers, `ci_services:edit` for inline Dockerfiles, `ci_pipelines:edit`), and any build pod, since all of them can reach p…
CI-06	MEDIUM	Inbound webhook endpoints read the whole body before any check (unauthenticated memory exhaustion)	Unauthenticated callers.
CI-07	MEDIUM	Build code controls workspace files that the collector and runner trust	Any stage code, including repository code.
CI-08	MEDIUM	`ci_pipelines:edit` effectively grants read access to every CI secret	Users with `ci_pipelines:edit` (or `ci_builds:run` on a pipeline that already references secrets) who hold no `ci_secrets:*` permission.
INT-02	MEDIUM	Changing an integration's URL keeps its stored secret, and the URL fields allow SSRF with partial response reflection	Users with `ticketing:manage` (Zoho, Jira, Jenkins), `registries:manage` or `vsphere:manage`.
INT-03	MEDIUM	The alert policy catalog returns webhook and Slack URLs to read-only users	Any user with `alerts:view` (default Viewer role).

ID	Severity	Finding	Who can exploit
INT-04	MEDIUM	Component health checks let Operators make the backend reach any host or port	Users with `components:create` or `components:update` plus `components:check` (default Operator role).
INT-05	MEDIUM	Security emails insert request-derived values into HTML without escaping	Unauthenticated callers, because lockout emails are triggered by failed sign-ins against a known username.
INT-06	MEDIUM	Ticket webhooks trust the whole payload, have no replay protection, and can name any stored target	A webhook caller holding the provider's inbound secret, and, through the ticket's own fields, external ticket authors.
K8S-06	MEDIUM	Approval with an empty voter email finalizes regardless of quorum (prior)	`change_bundles:manage` users whose account has no email, or any action token minted with an empty email.
K8S-08	MEDIUM	Server-side requests to user-supplied hosts: Helm repositories and cluster endpoints (prior SSRF)	`helm:install` holders (repo add, and `repositoryUrl` in install and dry-run payloads);
MCP-05	MEDIUM	Any ticket author can trigger unreviewed production changes; the only gate is the model's self-reported confidence	Anyone who can file a ticket.
MCP-06	MEDIUM	Per-cluster "Read only" and "Off" MCP rules are bypassed through CI tools that deploy indirectly	A token with `ci_builds:run` / `ci_builds:retry` on a service whose pipeline has a Deploy stage, or with `ci_pipelines:edit` + `apps:deploy` to add a new target in a clos…
MCP-08	MEDIUM	The remote build agent runs pipeline commands on its host; the pipeline can switch off its container boundary, and artifact paths are not confined	Anyone who can edit a pipeline routed to an agent runner (`ci_pipelines:edit`, in the UI or through the MCP pipeline tools).
PLAT-06	MEDIUM	A local branch holds Hermes runtime secrets (LLM OAuth refresh tokens, Telegram, ksa_)	Anyone who gets a copy of .git (a backup, a copied worktree, or a `push --all` / `--mirror`).
PLAT-08	MEDIUM	Version caps in requirements.txt block security fixes (gunicorn, flask-cors, cryptography, pytest)	A remote client, for request smuggling when the front proxy parses Transfer-Encoding differently from gunicorn;
PLAT-11	MEDIUM	The session JWT lives in localStorage and there is no CSP, so any XSS becomes full takeover	Anyone who manages to inject script: a dependency compromise, a rendering bug (APR-02 is one today, on a server-rendered page), or a malicious browser extension.
PLAT-12	MEDIUM	The backend runs as root and keeps a static copy of the service-account token	Anyone who gets code execution in the backend, for example through YAML, Helm, Excel or archive parsing, or the root-run kubectl, helm, tofu and git.
PLAT-13	MEDIUM	The admin control plane is on the external ingress class with no source restriction	Anyone on the network the external controller serves, possibly the internet.
PLAT-14	MEDIUM	The Werkzeug debugger PIN is committed, and the dev server binds 0.0.0.0 with debug on by default	Anyone on the same LAN or VPN as a developer running `python app.py`.
PROV-01	MEDIUM	Workload copy reads source-cluster Secret values without `secrets:reveal`	An authenticated user with `cluster_builds:create`, `cluster_builds:execute` and view access to a namespace on a source cluster.
PROV-02	MEDIUM	vCenter "Skip TLS verify" disables certificate validation for the provisioning account	An attacker able to intercept traffic between KubeSight and vCenter, once an admin turns on skipTlsVerify for the connection.
APR-10	LOW	The approval link base URL falls back to the request Host header	`deployment_requests:request` or `change_bundles:create`, only where `PUBLIC_BASE_URL` is unset and the Host header is not enforced.

ID	Severity	Finding	Who can exploit
APR-11	LOW	The Excel form import has no upload size limit	`apps:dryrun` (operator, cluster_admin).
APR-12	LOW	Any deployer can change any environment's scheduled departures	`apps:deploy`.
APR-13	LOW	Pipeline editors control self-approval on the CI Approval stage, and triggered builds record no starter	`ci_pipelines:edit` (cluster_admin), or a commit author on webhook- or cron-triggered builds.
APR-14	LOW	Approval group endpoints expose the user directory and can edit alert-routing groups	`deployment_requests:view` (operator, cluster_admin) to read the user directory;
AUTH-09	LOW	Formula injection in the audit CSV export, plantable by unauthenticated users	Unauthenticated, through the login username field.
AUTH-10	LOW	TOTP codes can be replayed	Someone who has the password and captures a single code (phishing relay, shoulder-surfing).
AUTH-11	LOW	CORS defaults to *	A malicious website, with limited effect.
AUTH-12	LOW	Admin-set passwords skip the password policy and the forced change	Holders of `users:update`.
AUTH-13	LOW	Login reveals account state before checking the password	Unauthenticated.
AUTH-14	LOW	The client IP is taken from a spoofable header	Unauthenticated.
AUTH-15	LOW	API tokens are unscoped and can mint more tokens	Whoever holds a leaked token whose owner has `api_tokens:manage`.
AUTH-16	LOW	Deleting a user relabels their audit history as automation	Holders of `users:delete` (admin by default).
CI-09	LOW	Merge-check pipelines run PR code automatically with no guard on Deploy or store-upload stages or on secrets	An external Bitbucket user who can push a branch and open a PR, but only if a pipeline editor has put secrets or a Deploy or store-upload stage into the merge-check pipel…
CI-10	LOW	Post-action webhooks are a blind SSRF from the backend pod	A user with `ci_secrets:manage` together with `ci_pipelines:edit`.
CI-11	LOW	Code-scan "Send" and email post actions send arbitrary text to arbitrary external addresses	Any user with `ci_artifacts:view` (in the default Viewer role) for Send, and `ci_pipelines:edit` for email post actions.
CI-12	LOW	Any CI user can use the stored Bitbucket credentials against any repository	`ci_services:view` holders (Viewer) for listing;
CI-13	LOW	Webhook secrets are accepted in the URL, and signed deliveries can be replayed	Anyone who sees ingress or proxy logs, or who captures a signed request.
CI-14	LOW	.dockerignore only excludes database files at the root, and the backend image is also the CI worker image	Any build stage (a pipeline editor can name this image as a stage image), but only if an image is ever built from a working copy.
INT-07	LOW	The component webhook token is returned to view-only users	Users with `components:view` (default Viewer role).
INT-08	LOW	Slack alert text inserts log content unescaped	Anyone who can make a monitored workload log text that matches a log-alert pattern, for example an external user of the application.
INT-09	LOW	The analysis revision is not validated before being passed to git	Users with `applications:analyze` (default Operator role).
INT-10	LOW	The mobile artifact workspace path can address any Jenkins path with the shared Jenkins credentials	Users with `mobile_apps:manage` (admin by default).
INT-11	LOW	Registry check-image puts the repository and tag into the request path unvalidated	Users with `registries:view`.

ID	Severity	Finding	Who can exploit
INT-12	LOW	Inbound ticket records are keyed by ticket id across providers	A holder of one provider's webhook secret.
K8S-10	LOW	Bundle diff runs without the `apps:diff` check	Bundle owners who only have `change_bundles:create`.
K8S-11	LOW	Kubernetes upgrade start is not under the cluster approval gate	`upgrades:start` holders (admin by default), and only when `KUBESIGHT_AUTO_UPGRADE=true`.
MCP-09	LOW	JSON-RPC batches are unbounded, and the MCP audit row hides repeated calls	Any authenticated token, including a viewer's.
MCP-10	LOW	Telegram approvals trust a changeable username, and an empty approver list lets anyone in the chat approve	Members of the approval chat, or anyone who can take over an approver's @username in a setup that uses an @channel.
MCP-11	LOW	API tokens have no scopes and no default expiry, so every MCP token is a full credential for its user	Not directly exploitable on its own;
MCP-12	LOW	Unknown-tool errors list every registered tool, including ones the admin switched off	Any authenticated token.
PLAT-16	LOW	Bitbucket PR pipelines run untrusted code, and repository secrets are probably reachable from them	Anyone who can push a branch and open a PR.
PLAT-17	LOW	Outdated and unverified tool binaries in the backend image	For the Helm issues, users with chart import or deploy rights who upload crafted charts.
PLAT-18	LOW	.gitignore gaps, and log files are tracked	No one directly;
PLAT-19	LOW	Weak dev credentials in the compose files, published on all interfaces	Anyone on the developer's LAN.
PROV-03	LOW	Default SSH host-key policy trusts the first key it sees (first-connect interception accepted silently)	An attacker who can intercept the first SSH connection from KubeSight to a node or bastion that uses the tofu policy.
PROV-04	LOW	The self-approval guard for destroys and VM requests is skipped when the request has no stored requester	Someone who can both request and approve, and only when the request was created with `requested_by_user_id = None`.
PROV-05	LOW	Authenticated host and port probe through the SSH test and host-key scan endpoints	An authenticated user with `ssh_credentials:manage`.
AUTH-18	INFO	The public dev key is always tried when decrypting	
AUTH-19	INFO	Write permissions apply across every namespace the user can see	
INT-13	INFO	The SMTP test is reachable with alerts:manage, while the equivalent route is admin-only	Users with `alerts:manage` (default Operator role).
PLAT-20	INFO	hermes-mcp-egress.yaml targets the wrong namespace	No one.
PLAT-21	INFO	The runtime config.js loader never runs in production, and uses new Function when it does	No one.
PLAT-22	INFO	A component heartbeat token is placed in the URL query string	Anyone who can read access logs.

6. Detailed findings

6.1 Authentication, sessions and RBAC

Scope reviewed

backend/app.py, backend/api/__init__.py (create_app: CORS, error handlers, /health, request size), frontend_static.py, runtime_config.py, auth_utils.py, decorators.py, passwords.py, secret_encryption.py; access.py, access_engine.py, access_rules.py, rbac_data.py, seed.py, serializers.py (user and role output); routes auth.py, users.py, roles.py, api_tokens.py, settings.py, audit_logs.py, access_rules.py, integrations.py, approvals.py, mcp.py (auth entry only); services auth_service, user_service, role_service, totp_service, settings_service, audit_service, audit.py; **Route sweep.** The real Flask app was built (TestingConfig, in-memory SQLite) and the decorator chain of all 645 registered routes was followed; 574 use `require_permission`, 15 `require_any_permission`, 14 `require_admin`, 31 `require_cluster_access`, 17 `require_namespace_access` and 3 `require_download_access`; 37 routes have no auth decorator and 13 have only `require_auth`; every one of those handlers was read; All 27 write routes guarded only by a `:view` permission were also read.

CRITICAL AUTH-01 Default accounts with public passwords and no MFA are re-created on every boot

Location backend/api/rbac_data.py:557-584; backend/api/seed.py:109-125; backend/api/__init__.py:350-352

Confidence Confirmed (PoC against an in-memory test app)

Who can exploit Unauthenticated: anyone who can reach the login page.

Description

`seed_defaults()` runs on every boot and creates `admin/admin123`, `operator/operator123` and `viewer/viewer123` whenever they are missing. These accounts default to `first_login_completed=True` and `mfa_enabled=False`, so a login gets a full access token with no second step, and the production boot checks do not look for them. Deleting them, the remediation the earlier audit recorded, does not stick: the next restart brings them back with the known passwords. Only disabling them or changing their passwords lasts.

Evidence

```
# seed.py
for spec in DEFAULT_USERS:
    user = User.query.filter_by(username=spec["username"]).first()
    if not user:
        user = User(username=spec["username"], password_hash=hash_password(spec["password"]), ...)
# PoC: DELETE /api/users/<viewer>/permanent -> 200; seed_defaults(); login viewer/viewer123 -> 200
```

Impact

An attacker with no credentials gets full admin: every cluster, exec, secret reveal, Helm, VM provisioning and minting API tokens.

Recommendation

- Seed a single bootstrap admin, only when the users table is empty.
- Give it a random or env-supplied password, with `must_change_password=True` and `first_login_completed=False`.
- Never re-create a user that was deleted.
- Make production refuse to boot while any active user still matches a default password.

HIGH AUTH-02 users:update and users:create are equivalent to Admin (H1, wider than reported)

Location backend/api/routes/users.py:71-110, 143-194; backend/api/services/user_service.py:145-222; backend/api/routes/access_rules.py

Confidence Confirmed (PoC)

Who can exploit A non-admin whose role holds `users:update`, `users:create` or `users:manage`. Only Admin has these by default, so this affects custom roles such as a "helpdesk" role.

Description

Nothing compares the target's privileges with the actor's, which opens four paths:

1. PUT your own `roleId` to admin. The guard only blocks removing your own rights, never adding them.
2. PUT `password` or `email` on any admin. Either set the admin's password (no policy applies), call reset-mfa, then log in and enrol your own TOTP; or change the admin's email to yours and call resend-temporary-password or

force-password-reset, which emails the temporary password to you.

3. POST /api/users with roleId=admin and an email you control. Outside production, the temporary password is also returned in the response.
4. Use the access-rules routes to grant any user, yourself included, any cluster scope.

Evidence

PoC: role {users:view, users:update} → PUT self roleId=admin → 200, then GET /api/audit-logs → 200. Role {users:view, users:create} → POST /api/users roleId=admin → 201, temporaryPasswordRevealed=True.

Impact

Escalation to Admin, and takeover of any admin account, including MFA-protected ones.

Recommendation

- Only allow granting what the actor holds: the target role or rule permissions must be a subset of the actor's.
- Only admins may modify privileged users.
- Remove `password` from PUT /users.
- Require step-up authentication for any change to a privileged account.

HIGH

AUTH-03 roles:manage can give its own role every permission

Location backend/api/services/role_service.py:182-189, 233-278; backend/api/access_engine.py:102-110

Confidence Confirmed (PoC)

Who can exploit Holders of `roles:manage`, or of `users:manage`, which implies it.

Description

`_actor_retains_management` only stops you removing management permissions from your own role; adding permissions is unchecked. `_compute_is_admin` treats any role that holds every permission key as admin. Creating a role with any set of permissions is also unchecked.

Evidence

PoC: role {roles:view, roles:manage} → PUT /api/roles/<own role>/permissions with every key → 200. `/api/auth/me` then reports isAdmin=True, and POST /api/auth/tokens {user_id:1} → 201, which is a token minted as the real admin.

Impact

Escalation to Admin, plus impersonation of other users through API tokens.

Recommendation

Refuse any permission key the actor does not hold, and make editing your own role admin-only.

HIGH

AUTH-04 The cluster-access check can be bypassed by sending a second cluster parameter

Location backend/api/decorators.py:159-196; handlers in backend/api/routes/dashboard.py:11-15, upgrades.py:33-35, 59-63, 87-91, helm.py:111-113, deployment_requests.py:16-18

Confidence Confirmed (PoC; decorator order re-verified at backend/api/decorators.py:159-168)

Who can exploit A logged-in user scoped to at least one cluster who also has the route's permission. Default roles: viewer and operator (`overview:view`, `upgrades:precheck`) and cluster_admin (`helm:install`); custom roles with `upgrades:start`.

Description

When the cluster is not in the URL path, `require_cluster_access` reads `args.cluster`, then `args.clusterId`, then `json.clusterId`, then `json.cluster_id`, and lets the request through if none is present. The handlers read the cluster from a different place:

- dashboard and upgrade info read `args.clusterId`;
- upgrade precheck and start read `json.clusterId`;
- Helm add-repo reads `json.clusterId` or `json.cluster`;
- deployment-request create reads `json.cluster_id` or `json.clusterId`.

So the decorator validates one cluster while the handler acts on another, and none of the services behind these handlers re-checks cluster access.

Evidence

```
user scoped to prod-us-east only:
GET /api/dashboard/summary?clusterId=staging-eu-west                      -> 403
GET /api/dashboard/summary?cluster=prod-us-east&clusterId=staging-eu-west  -> 200 (staging data)
POST /api/upgrades/precheck?cluster=prod-us-east  {clusterId: staging-eu-west} -> 200
```

Impact

A user can read another cluster's nodes, versions, capacity and health, add a Helm repo to another cluster's config, and file deployment requests against clusters they are not assigned to. Custom roles with `upgrades:start` can also start a Kubernetes upgrade on such a cluster.

Recommendation

- Resolve the cluster once in the decorator, store it on `g`, and have handlers read only that value.
- Reject requests that carry conflicting cluster values.
- Return 400 when the cluster is missing instead of letting the request through.
- Better still, move these routes to `/<cluster_id>/...` paths.

MEDIUM

AUTH-05 Anyone can lock any account, up to an admin-only unlock

Location backend/api/services/auth_service.py:293-380, 404-410, 529-531

Confidence Confirmed (PoC)

Who can exploit Unauthenticated; only a username or email is needed.

Description

- Five wrong passwords lock the account for 15 minutes.
- The failure counter is reset only by a successful login, so after each lock expires a single wrong attempt locks the account again.
- The third lock within 24 hours sets `requires_admin_unlock`.
- The lock check runs before the password check, so the real owner is locked out too.
- There is no per-IP throttling, and every attempt writes an audit row.

Evidence

PoC: 5 wrong → 423; after expiry 1 wrong → 423; after expiry 1 wrong → requires_admin_unlock=True; correct password → 423.

Impact

Denial of service against any account. If every admin is locked out, recovery needs direct database access.

Recommendation

- Throttle per IP and per username instead of hard-locking, and reset the counter when a lock expires.
- Do not escalate to admin-only unlock on unauthenticated failures alone.
- Provide a break-glass unlock (CLI or env), and rate-limit audit rows from failed logins.

MEDIUM

AUTH-06 API tokens keep working on locked accounts

Location backend/api/auth_utils.py:234-252; backend/api/decorators.py:326-341; backend/api/services/user_service.py:421-455

Confidence Confirmed (PoC)

Who can exploit Anyone holding a `ksa_` token of a user that an admin locked (for example a compromised account) or that was auto-locked.

Description

Token login checks only `is_active` and expiry, never `requires_admin_unlock` or `locked_until`. Locking an account revokes browser sessions only and leaves API tokens alone by design.

Evidence

PoC: lock viewer → password login 423, but the viewer's ksa_ token still gets 200 on /api/auth/me and /api/clusters.

Impact

Locking an account does not cut off its automation or MCP credentials.

Recommendation

Refuse locked users in token login (and in JWT login), and optionally deactivate their tokens when an admin locks them.

MEDIUM AUTH-07 The onboarding token still works after onboarding and can change the password

Location backend/api/services/auth_service.py:643-673; backend/api/routes/auth.py:47-57; backend/api/auth_utils.py:108-120

Confidence Confirmed (PoC)

Who can exploit Anyone who obtains a user's onboarding token within its 30-minute lifetime.

Description

`change_temporary_password` does not check `must_change_password`, and nothing revokes interim tokens. After MFA enrolment, the same onboarding token can still set a new password without the current password and without TOTP.

Evidence

PoC: onboarding complete → reuse the onboarding token on /first-login/change-password → 200 → the new password logs in.

Impact

Someone holding the token can silently change the password and lock the real user out.

Recommendation

Require the `password_change` stage, and revoke interim tokens (bump `token_version`) once the password change and TOTP verification complete.

MEDIUM AUTH-08 No request size limit, including on unauthenticated routes

Location backend/api/__init__.py:298-403; k8s/ingress.yaml:12

Confidence Confirmed in code (no load test run)

Who can exploit Unauthenticated, via /api/auth/login, inbound webhooks, CI hooks and similar routes.

Description

`get_json()` reads and parses the entire body. There is no `MAX_CONTENT_LENGTH`, the ingress allows 512m, and the default is one gunicorn worker. CI-06 shows the same gap on the CI webhook routes.

Impact

A memory-exhaustion DoS gets the backend pod OOM-killed.

Recommendation

Set `MAX_CONTENT_LENGTH` to 2-10 MB, give the upload routes their own larger per-route limits, and lower the ingress body size.

LOW AUTH-09 Formula injection in the audit CSV export, plantable by unauthenticated users

Location backend/api/routes/audit_logs.py:209-222; backend/api/services/auth_service.py:354-361

Confidence Confirmed (PoC)

Who can exploit Unauthenticated, through the login username field. The victim is an `audit:view` user who opens the export in Excel.

Description

A failed login stores the raw identifier as the audit target, and the CSV export writes it unescaped, so a value starting with = becomes a live spreadsheet formula.

Recommendation

Prefix cells starting with `= + - @` or a tab or carriage return with `'`, and truncate the identifier (the column is VARCHAR(255), and a longer value errors on Postgres).

LOW AUTH-10 TOTP codes can be replayed

Location backend/api/services/totp_service.py:59-69

Confidence Confirmed (PoC)

Who can exploit Someone who has the password and captures a single code (phishing relay, shoulder-surfing).

Description

Codes are checked with `valid_window=1` (about 90 seconds), and the last used time step is not recorded, so the same code was accepted twice.

Recommendation

Store the last used step per user, and reject any step that is not newer.

LOW AUTH-11 CORS defaults to *

Location backend/api/__init__.py:289-295; backend/api/runtime_config.py:99-119

Confidence Confirmed

Who can exploit A malicious website, with limited effect. The session token lives in localStorage and is sent as a header, not a cookie, so another site cannot borrow a session.

Description

This was a Medium in the earlier audit. `CORS_ORIGINS` defaults to `*`, and the production boot checks do not refuse it. This becomes serious the moment any credential becomes ambient, such as a session cookie.

Recommendation

Refuse `*` in production, and scope CORS to /api/*.

LOW AUTH-12 Admin-set passwords skip the password policy and the forced change

Location backend/api/routes/users.py:107-109

Confidence Confirmed (code)

Who can exploit Holders of `users:update`.

Description

PUT /api/users `password` accepts any string, with no policy check, no `must_change_password` and no notice to the user. It is one of the main levers in AUTH-02.

Recommendation

Remove the field, or apply the policy, force a change at next login and notify the user.

LOW AUTH-13 Login reveals account state before checking the password

Location backend/api/services/auth_service.py:514-537

Confidence Confirmed (code)

Who can exploit Unauthenticated.

Description

A disabled account returns 403 "disabled" and a locked account returns 423 with lock details, both before the password is checked. An unknown user skips password hashing, so its response time differs from a real user's.

Recommendation

Check the password first (against a dummy hash for unknown users), and return a generic 401.

LOW AUTH-14 The client IP is taken from a spoofable header

Location backend/api/services/auth_service.py:143-149

Confidence Confirmed (code)

Who can exploit Unauthenticated.

Description

`_client_ip` trusts the first `X-Forwarded-For` value, and that IP goes into login and lock audit rows and into the "you signed in" email.

Recommendation

Use `ProxyFix` with the real proxy hop count.

LOW AUTH-15 API tokens are unscoped and can mint more tokens

Location backend/api/auth_utils.py:234-252; backend/api/routes/api_tokens.py:54-112

Confidence Confirmed (code)

Who can exploit Whoever holds a leaked token whose owner has `api_tokens:manage`.

Description

A token carries the owner's full permissions, and a leaked token can create a new token with no expiry, so revoking the leaked one does not end access. MCP-11 covers the same lack of scopes from the agent side.

Recommendation

Add scopes, forbid minting tokens or administering users and roles when the caller signed in with an API token, and require an expiry.

LOW AUTH-16 Deleting a user relabels their audit history as automation

Location backend/api/services/user_service.py:528-531; backend/api/routes/audit_logs.py:161-166

Confidence Confirmed (code)

Who can exploit Holders of `users:delete` (admin by default).

Description

Deleting a user clears the actor on their audit rows, which then display as "KubeSight automation".

Recommendation

Store the actor's username on each audit row when it is written, make the audit log append-only, and disable users who have history instead of deleting them.

INFO AUTH-18 The public dev key is always tried when decrypting

Location backend/api/secret_encryption.py:174-191

Description

Rows encrypted before a real key was set stay readable from any DB backup using the public dev key, and nothing re-encrypts them.

Recommendation

Run a one-off re-encryption pass, then stop trying the dev key in production.

INFO AUTH-19 Write permissions apply across every namespace the user can see

Location backend/api/access_engine.py:316-376; backend/api/services/resource_actions_service.py:444-475

Description

Per-cluster rules only control what a user can see. Write actions check only the role permission plus that visibility, so `apps:deploy` works on every namespace the user can see. This appears to be by design: the rules editor offers only view actions.

6.2 Kubernetes cluster access and mutation

Scope reviewed

backend/api/routes/clusters.py, helm.py, change_bundles.py, upgrades.py (start/job routes), inventory.py (deploy yaml/image/wizard apply paths, as callers of deployment_service); backend/api/decorators.py, access_engine.py, access.py, cluster_access.py, rbac_data.py (role contents); backend/api/k8s_provider.py (runner, `_refuse_unsafe_kubectl_args`, `resolve_cluster_access`, events), k8s_names.py, kubeconfig_vault.py, cluster_store.py (host validation); backend/api/services/resource_actions_service.py, change_bundle_service.py, change_bundle_executor.py, deployment_service.py, helm_service.py, helm_chart_template_service.py (archive extraction), cluster_connection_service.py (update path), logs_service.py (per-pod checks), deployment_request_service.py (gate helpers); Not reviewed in depth: upgrade_executor.py internals, kube_direct.py, generator internals, topology/dashboard read paths (no write surface).

HIGH

K8S-01 Viewer-level users can stage arbitrary manifests that the executor applies with KubeSight's credentials, with kind restrictions lifted

Location backend/api/services/change_bundle_service.py:780-805, 1058-1072; backend/api/services/change_bundle_executor.py:90-98, 204-216; backend/api/rbac_data.py:356

Confidence Confirmed

Who can exploit Any logged-in user with `change_bundles:create`, which the built-in viewer, operator and cluster_admin roles have. Treat as Critical on any installation where a cluster, or the global default, requires 0 approvals.

Description

Staging an apply-mode item checks only that the user can view the namespace (and only when a namespace is present); cluster access is never checked. Validation runs with `preview_mode=True`, which turns `BLOCKED_CLUSTER_KINDS` into warnings. If the cluster needs 0 approvals (a per-cluster override or the global setting), submit approves the bundle automatically, and the scheduler then applies it without checking the requester's permissions again. On approval-gated clusters, one approver's click applies cluster-scoped or privileged kinds that a read-only user wrote, and the approvals inbox names only each item's primary resource, so the approver does not see the other documents of a multi-document item. The approvals review reproduced this at runtime.

Evidence

```
# Staging only requires access to the target namespace — not deploy rights — ...
if user is not None and namespace and not can_access_namespace(user, cluster_id, namespace):
...
if required == 0:
    bundle.status = "approved"
```

Impact

On clusters set to 0 approvals, a read-only account can change cluster state, including kinds that grant privileges, using KubeSight's credentials (usually cluster-admin).

Recommendation

- Require each action's own permission (`apps:deploy` / `apps:delete`) to stage every action type. Today `_STAGING_REQUIRES_PERMISSION` covers only Helm; extend it to all of `ACTION_TYPES`.
- Never auto-approve for requesters who lack that permission.
- Validate with the requester as `user` and `preview_mode=False` at staging and execution, so cluster-scoped kinds stay admin-only.
- Re-check the requester's current permissions in `execute_bundle`.

HIGH

K8S-02 Staged Secret values and Helm values are stored and returned unredacted to any bundle reader

Location backend/api/services/change_bundle_service.py:216-234, 546-557, 819; backend/api/routes/change_bundles.py:48-56, 237-256; backend/api/rbac_data.py:359

Confidence Confirmed

Who can exploit Anyone with `change_bundles:view` (default viewer role), for bundles on any cluster, whatever their cluster or namespace grants.

Description

The `yamlPreview` of `edit_secret` and `apply_yaml` items comes from `sanitize_for_apply`, which keeps `data`/`stringData`. The raw input YAML is also kept in `newPayload.input`, and queued Helm items keep the raw `valuesYaml`. `GET /api/change-bundles/<id>` returns all of this to the requester or to any holder of `change_bundles:view`/`manage`. It neither checks `secrets:reveal` nor scopes by cluster.

Evidence

```
yaml_content = sanitize_for_apply(yaml_content)              # values kept
"newPayload": _payload_for_display(item.new_payload_json),    # only strips chart archive
"yamlPreview": item.yaml_preview,
```

Impact

Secret values that the YAML view hides behind `secrets:reveal` can be read by viewers through the bundle API, across clusters they were never granted.

Recommendation

- In `serialize_item`, redact Secret values (`redact_secret_yaml`) and Helm values (`_mask_sensitive_values`) unless the reader has `secrets:reveal`.
- Never return the raw `input.yaml` or `input.helm.valuesYaml`.
- Limit reads and lists to bundles on clusters and namespaces the reader can access.

HIGH

K8S-03 `clusters:update` can repoint any cluster's stored credentials to another API endpoint; management routes are not cluster-scoped

Location backend/api/routes/clusters.py:264-322, 325-361, 364-387; backend/api/services/cluster_connection_service.py:210-245; backend/api/cluster_store.py:56-60

Confidence Likely. To confirm, check that `build_cluster_kubeconfig` keeps the existing `users[]` credentials when only the host changes.

Who can exploit Users with `clusters:update` (plus `clusters:test` to start a connection). They do not need to be admins or to have access to the target cluster.

Description

`PUT /api/clusters/custom/<ref>` does not call `can_access_cluster`. For manually configured clusters, changing only host, port or protocol rebuilds the kubeconfig from the stored one, credentials included, pointed at the new endpoint. `validate_host` accepts any host, so the next connection test or read sends the stored token or certificate to that endpoint. DELETE and `/test` are also unscoped.

Evidence

```
existing = read_kubeconfig_file(cluster.id)
rendered, _ = build_cluster_kubeconfig(kubeconfig_content=existing, host=host, port=port, ...)
```

Impact

Cluster credentials, often cluster-admin, can leak to a user whose role was only meant to edit connection settings. Scoped users can also edit or delete clusters they were not granted.

Recommendation

- Enforce `can_access_cluster` (or admin-only) on every `/custom/<ref>` route.
- When the endpoint changes, require the credentials to be entered again, or allow only admins to change endpoints.
- Audit the old and new endpoint.

HIGH

K8S-04 YAML/image/wizard apply relies on a kind denylist; namespace-scoped deployers can exceed their namespace grant

Location backend/api/services/deployment_service.py:21-45, 218-267, 639-765

Confidence Confirmed in code. The real impact depends on the cluster's admission policy (Medium where tenant namespaces enforce Pod Security Admission).

Who can exploit `apps:deploy` holders limited to certain namespaces.

Description

Only nine kinds are blocked for non-admins; any other kind only produces a warning and is applied. Pod specs are not checked for privileged mode, host namespaces, hostPath or service-account choice. Cluster-scoped kinds that are not on the denylist are applied with KubeSight's credentials, outside the user's namespace scope.

Evidence

```
if kind not in ALLOWED_NAMESPACED_KINDS and kind not in BLOCKED_CLUSTER_KINDS:
    warnings.append(f"{kind}/{name} is not in the default allowlist")
```

Impact

A deployer limited to some namespaces can reach node-level or cluster-wide effects, which defeats KubeSight's per-namespace RBAC.

Recommendation

- Use an allowlist for non-admins and refuse cluster-scoped kinds for them.
- Reject pod specs with host-level settings unless the user is an admin.
- Recommend Pod Security Admission on tenant namespaces as a second layer.

HIGH

K8S-05 Pod exec (UI and the kubesight_pod_exec MCP tool) bypasses the cluster approval gate and has no dedicated permission (prior pods:exec)

Location backend/api/services/resource_actions_service.py:84-86, 619-694; backend/api/routes/clusters.py:899-919

Confidence Confirmed

Who can exploit `apps:deploy` holders with access to the namespace, on any cluster including approval-gated ones, and any MCP token with that permission (including Hermes if its token has it). The MCP "Read only" cluster mode does block exec; the approval gate does not.

Description

`exec_in_pod` reuses the restart access checks but never calls `gate_or_queue`, so the command runs immediately. There is no `pods:exec` permission, and exec exposes the pod's Secrets and environment without `secrets:reveal`.

Evidence

```
denied = _check_resource_restart_access(user, cluster_id, namespace, "pod", pod_name)
...
args += ["--", "sh", "-c", command]   # no approval gate on this path
```

Impact

On clusters where every change needs approval, an exec can still kill PID 1, rewrite data, or read mounted secrets and service-account tokens and call the Kubernetes API, with no approval and drivable by an agent.

Recommendation

- Add a separate `pods:exec` permission.
- On gated clusters, call `check_cluster_change_allowed(..., action="exec")` and refuse (not queue) unless the user has a live approved request.
- Consider shipping `kubesight_pod_exec` switched off by default.
- Keep the per-command audit.

MEDIUM

K8S-06 Approval with an empty voter email finalizes regardless of quorum (prior)

Location backend/api/services/change_bundle_service.py:1163-1165, 1196, 1219

Confidence Confirmed

Who can exploit `change_bundles:manage` users whose account has no email, or any action token minted with an empty email.

Description

`decide_bundle` passes the actor's email. If it is empty, `record_vote` finalizes the bundle as approved and never checks `required_approvals`.

Evidence

```
if not email:
    _finalize(bundle, "approved" if action == "approve" else "rejected", actor=actor, reason=reason)
```

Impact

One approver satisfies an N-of-M approval policy.

Recommendation

Record in-app votes by user id and always run the quorum check. Refuse approvals from accounts with no identity key.

MEDIUM

K8S-08 Server-side requests to user-supplied hosts: Helm repositories and cluster endpoints (prior SSRF)

Location backend/api/services/helm_service.py:106-115, 310-321, 639-651; backend/api/routes/helm.py:96-111; backend/api/cluster_store.py:56-60

Confidence Confirmed

Who can exploit `helm:install` holders (repo add, and `repositoryUrl` in install and dry-run payloads); `helm:view` holders (`/api/helm/template`); `clusters:add` and `clusters:update` holders (the cluster host).

Description

Repo URLs are checked only for scheme, then fetched by the backend's Helm, and Helm's error text is returned to the caller. Repos are added to one Helm config shared by all users and clusters, so a repo name registered by one user is used by everyone's later installs. Cluster hosts accept internal addresses.

Impact

The backend can be used to probe internal services, and one user's repo can shadow a name another user's install relies on.

Recommendation

- Keep an admin-managed allowlist of repositories, and stop accepting `repositoryUrl` in install, dry-run and template payloads.
- Give each request its own Helm repository config and cache, or make repo registration admin-only.
- Return generic fetch errors.
- Optionally refuse link-local and metadata address ranges.

LOW

K8S-10 Bundle diff runs without the `apps:diff` check

Location backend/api/services/change_bundle_service.py:933-935

Confidence Confirmed

Who can exploit Bundle owners who only have `change_bundles:create`.

Description

`diff_yaml(None, ...)` skips both the `apps:diff` check and the namespace check at diff time.

Impact

Limited reads of live object state beyond what resource-scoped rules allow.

Recommendation

Pass the requesting user to `diff_yaml`.

LOW

K8S-11 Kubernetes upgrade start is not under the cluster approval gate

Location backend/api/routes/upgrades.py:81-104

Confidence Likely. No gate call was found in upgrade_service.py, upgrade_executor.py or upgrade_jobs.py; confirm in `run_start`.

Who can exploit `upgrades:start` holders (admin by default), and only when `KUBESIGHT_AUTO_UPGRADE=true`.

Description

Starting an upgrade needs only the typed confirmation, not an approved request.

Impact

The most disruptive cluster change skips the approval rule that every other write path enforces.

Recommendation

Call `check_cluster_change_allowed` in `run_start`.

6.3 Native CI platform

Scope reviewed

Routes: ci.py, ci_agent.py, ci_worker.py, ci_webhooks.py, ci_merge_checks.py, ci_schedules.py, ci_shared_pipelines.py, ci_assist.py; Services: backend/api/services/ci/** (engine, pipelines, catalog, secrets, logs, artifacts, workspace, webhook_triggers, schedules, agents, runners/kubernetes.py, cache, cache_layout, deploy_config, deploy_stage, deployment_links, shared_pipelines, approval_stage, approval_config, post_actions, code_scan*, scan_stage, store_upload_*, build_inputs, merge_checks/*, source/*) and backend/api/services/ci_assist/**; Models: models_ci*.py, models_merge_checks.py; Infra: k8s/ci-*.yaml, k8s/ingress.yaml, backend/Dockerfile, backend/.dockerignore, Dockerfile.ci-imagetools, rbac_data.py; All code added since 2026-09-26: webhook triggers, Deploy stage, shared pipelines, deployment links and templates, scan/approval/store-upload/parallel stages, schedules, cache concurrency, merge-check linters, BuildKit failover.

HIGH

CI-01 One shared cache volume, mounted whole into every build, lets any build poison other services' builds

Location backend/api/services/ci/runners/kubernetes.py:1908-1925, :2268-2276; k8s/ci-backend-config.yaml (CI_CACHE_CLAIM_NAME: ci-cache); backend/api/services/ci/merge_checks/linters.py:81-126; backend/api/services/ci/cache_layout.py:338

Confidence Confirmed

Who can exploit Anyone whose code runs in any build: users with `ci_pipelines:edit`, users with `ci_builds:run` building a branch they control, and external people who can push a branch or open a PR on any Bitbucket repo that KubeSight builds through a push webhook or merge checks.

Description

All services share one hand-made NFS claim. `_mounts()` mounts the whole claim (no `subPath`) at `/kubesight-cache` and `/cache` in every stage container. All containers run as uid/gid 65532 with `fsGroup` 65532, so the `/kubesight-cache/<slug>` separation is only a naming convention. Other services' builds execute or trust content from this volume:

- every `*.gradle` file in `<slug>/gradle/init.d`;
- the `CARGO_HOME` config and the Maven repository;
- the `node-modules/<key>.tar` archives, restored without an integrity check;
- the merge-check linters (Ruff, PMD), installed once into `_shared/tools` and then run by path by every service.

Evidence

```
# runners/kubernetes.py:1920
mounts.append({"name": "cache", "mountPath": CACHE_MOUNT_PATH})        # whole volume
# merge_checks/linters.py:81,113-125
'TOOLS_DIR="${KUBESIGHT_SHARED_CACHE_DIR:-${KUBESIGHT_CACHE_DIR:-/tmp}}/tools"'
'RUFF="$RUFF_DIR/bin/ruff"'
```

Impact

A low-trust build can run code inside a high-value service's builds, which hold its secrets. It can also tamper with outputs that service's Deploy or store-upload stage then ships, and post forged "pass" merge-check verdicts on other repositories' PRs.

Recommendation

- Mount `subPath: <slug>` per service, and mount `_shared` read-only.
- Populate shared tools only from a trusted Job, and verify their sha256 before running them.
- Use separate claims across trust boundaries.
- Never run binaries or init scripts from a location builds can write.

HIGH

CI-02 Any earlier stage can run code inside the image-build stage, which holds the registry push credentials

Location backend/api/services/ci/runners/kubernetes.py:495-505, :538-546, :2120-2190

Confidence Confirmed

Who can exploit Any code that runs before a `container_image` stage. Repository code (tests, npm scripts, Gradle files) counts, so this includes repo committers and holders of `ci_builds:run` or `ci_pipelines:edit`.

Description

`_wrap_stage_script` sources `/workspace/.kubesight/build.env` in every stage, image stages included. Any stage can write that file, because the workspace is shared and every container runs as the same uid. Only the image stage mounts `/kubesight-docker/config.json`, which holds the registry connection's decrypted credentials. When the scan gate is armed, the same container also runs `trivy` and `crane push`.

Evidence

```
_LOAD_BUILD_ENV = (f'export KUBESIGHT_ENV={_BUILD_ENV_FILE}\n'
                   f'if [ -s "$KUBESIGHT_ENV" ]; then . "$KUBESIGHT_ENV"; fi\n')
+ f"(\nset -e\n{_LOAD_BUILD_ENV}{body}\n)\nEC=$?\n"   # body = buildctl/trivy/crane
```

Impact

The registry push credential (usually one deployer account shared by many services) can be stolen and used to overwrite any service's image, a supply-chain compromise. Code in build.env can also shadow the `trivy` command and so switch off the image scan gate.

Recommendation

- Do not source build.env in image or scan stages. Read only the `NAME=value` lines the tag or name template uses, as data.
- Push from a container that reads no workspace control files.
- Call tools by absolute path.

HIGH

CI-03 The Deploy stage's authorization stamp covers the target, not the code that gets deployed

Location	backend/api/services/ci/deploy_config.py:423-456; backend/api/services/ci/pipelines.py:837-906 (stamp carried over at :864); backend/api/services/ci/deploy_stage.py:750-812; backend/api/services/ci/deployment_links.py:348-366; backend/api/services/ci/shared_pipelines.py:345; backend/api/services/ci/store_upload_config.py:109-125
Confidence	Confirmed
Who can exploit	Users with `ci_pipelines:edit` and/or `ci_services:edit` who lack `apps:deploy` or access to the namespace. A weaker form is open to `ci_builds:run` holders, who can pick any branch. CI permissions are global, and `catalog.get_service` does no per-service check.

Description

The stamp is kept on every later save while `signature()` is unchanged, and the signature covers only the target fields (cluster, namespace, deployment, container, fixed image, manifest/template). With `image` empty, which is the default, none of the following is covered, so all can change without re-authorization:

- the repo URL and credential;
- the inline Dockerfile;
- the other stages;
- IMAGE_NAME;
- the branch.

The same gap appears elsewhere:

- **Shared pipelines.** `attach()` needs only `ci_pipelines:edit`, so any service can use a shared pipeline whose fixed Deploy stage someone else stamped.
- **Deployment links.** `update_link()` lets anyone with `ci_services:edit` who can merely see the namespace change the container, environment, template and template answers, including which existing Secret is mounted. `authorized_by` is not re-checked.
- **Store-upload stages.** The admin stamp covers the target, not the binary.

Evidence

```
material = {"clusterId":..., "namespace":..., "deploymentName":..., "containerName":...,
            "image": config.get("image") or "", "createIfMissing":..., "manifest":...}
kept = stored.get(deploy_config.signature(config))
if kept: config["authorizedBy"] = dict(kept)
```

Impact

Someone with no deploy rights can replace the code running in, for example, prod/payments. The change goes through `apply_yaml` as the original authorizer, and the only remaining control is the cluster approval gate where it is enabled.

Recommendation

- Stamp over a hash of the whole pipeline plus the service's source settings (repo URL, credential, Dockerfile, working directory), and require a deployer to re-authorize after any change.
- Require deploy rights to `attach()` a pipeline that has fixed Deploy or store-upload stages.
- Clear the link's `authorized_by` when its fields change.
- Optionally restrict stamped deploys to protected branches.

HIGH

CI-04 `ci_runners:manage` can mount any node path or NFS export into untrusted build pods

Location backend/api/routes/ci.py:1488-1510; backend/api/services/ci/cache.py:422-553; k8s/ci-cache-rbac.yaml

Confidence Confirmed for the mount. Likely for kubelet's recursive fsGroup re-ownership of a local PV; confirm on a scratch node.

Who can exploit Users with `ci_runners:manage`, a CI-admin permission rather than KubeSight Admin.

Description

`POST /api/ci/cache/volume` creates a PV from the request, either `backing: "local"` with any absolute path on any node or any NFS server and export, and saves that claim as the build cache. The backend's ClusterRole grants PV create. From then on, every build pod mounts the volume read-write. Pod `fsGroup: 65532` with `OnRootMismatch` makes kubelet re-own a local volume, and the "clean" maintenance Job deletes paths under it.

Evidence

```
elif backing_type == "local":
    path = _require_path(payload.get("path"), "The directory on the node")   # only absolute, no '..'
    source = {"local": {"path": path}, "nodeAffinity": {...}}
```

Impact

A path such as `/` or `/etc/kubernetes` gives build code read/write access to the node's filesystem, and potentially root on the node and then the cluster. An NFS export instead exposes other applications' data to reading, writing or deletion.

Recommendation

- Remove `local` backing, or limit backings to an operator allowlist set through an environment variable.
- Require Admin to create the volume.
- Never apply fsGroup to a directory that is not dedicated to the cache.

MEDIUM

CI-05 Shared BuildKit daemon runs without a process sandbox, unconfined, with no client authentication

Location k8s/ci-buildkitd.yaml:27-35, :132, :160-176

Confidence Likely. The manifest documents this as an accepted cost; cross-build `/proc/<pid>/root` access is not yet demonstrated.

Who can exploit Anyone who controls a built Dockerfile (repo committers, `ci_services:edit` for inline Dockerfiles, `ci_pipelines:edit`), and any build pod, since all of them can reach plaintext gRPC on port 1234.

Description

The daemon runs with `--oci-worker-no-process-sandbox`, Unconfined seccomp and AppArmor, and `allowPrivilegeEscalation: true`, in a privileged PSA namespace. It is shared by all services and has no mTLS. Concurrent builds' RUN steps share the daemon's PID namespace and uid, so they can read each other's `/proc/<pid>/environ` and `/proc/<pid>/root`, signal or ptrace the daemon, and write its content store.

Impact

Source and build-time secrets can be stolen across services, and other services' image layers can be poisoned. This compounds CI-01 and CI-02.

Recommendation

Run buildkitd per trust group, as the manifest itself suggests, or use daemonless BuildKit under gVisor or Kata. Enable TLS client authentication. Also tighten the daemon's NetworkPolicy: its egress rule (k8s/ci-buildkitd.yaml:283-300) lists ports but has no `to:`, so build steps can reach any internal IP on ports 443, 8443, 8082, 5000 and 9443, including backend-service:5000 directly, even though the comment says only registries and DNS are reachable. Add an ipBlock that excludes RFC1918, plus the registry CIDRs.

MEDIUM

CI-06 Inbound webhook endpoints read the whole body before any check (unauthenticated memory exhaustion)

Location backend/api/routes/ci_merge_checks.py:294-311; backend/api/routes/ci_webhooks.py:211-222; k8s/ingress.yaml:12

Confidence Confirmed in code; not load-tested

Who can exploit Unauthenticated callers. No valid slug, id or secret is needed, because the body is read first.

Description

The merge-check inbound route calls `request.get_data(cache=True)` and then `get_json()` before any size check, lookup or secret check. The CI hook checks Content-Length, but a chunked request has none, and the full body is read before the post-read length check. There is no `MAX_CONTENT_LENGTH`, and the ingress allows 512m.

Evidence

```
provided = request.headers.get("X-KubeSight-Secret") or request.args.get("secret", "")
raw_body = request.get_data(cache=True) or b""      # before any check
```

Impact

A few concurrent large POSTs exhaust memory on the backend's few gunicorn workers and take down the whole API, approvals and deploys included.

Recommendation

- Set `MAX_CONTENT_LENGTH` globally, with a larger limit only on the upload routes.
- Reject before reading when the length is missing or too big, and stream-limit reads.
- Lower the ingress body size on these paths.

MEDIUM

CI-07 Build code controls workspace files that the collector and runner trust

Location backend/api/services/ci/runners/kubernetes.py:433-475, :3160-3188; backend/api/routes/ci_worker.py:84-178; backend/api/services/ci/deploy_stage.py:779-811

Confidence Confirmed for the code paths. Likely for end-to-end token reuse, which was not executed.

Who can exploit Any stage code, including repository code. Retrieving the leaked token also needs `ci_artifacts:view`, which the Viewer role has.

Description

1. **Image record.** The collector takes the recorded image from the build-writable `/workspace/.kubesight/image-meta-N.json`. The Deploy stage deploys the newest container-image artifact, so a later stage can point the deploy at any image in a linked registry, which also sidesteps the scan gate.
2. **Callback token.** The artifact glob uses `os.path.isfile`, which follows symlinks, so the build's `KUBESIGHT_CALLBACK_TOKEN` can end up uploaded as an artifact. With that token, `POST /artifacts` accepts an arbitrary `uri`, and `POST /meta` accepts an arbitrary `commitSha`. KubeSight then posts the build status on that commit, which could satisfy a Bitbucket merge check for unreviewed code.
3. **Result markers.** The stage verdict is the first `[kubesight-exit]` line among the last five log lines, so a failing command that prints `[kubesight-exit] 0` is reported as passed.

Evidence

```
matches = [p for p in glob.glob(pattern, recursive=True) if os.path.isfile(p)]  # follows symlinks
"uri": meta.get("image.name") or image.get("uri") or "",                        # build-writable
```

Impact

The scan and quality gates can be bypassed, an image other than the one built can be deployed, and commit statuses can be forged.

Recommendation

- Record the image `uri@digest` from the image stage only, and have the Deploy stage use that.

- In the collector, resolve each match with `realpath`, confine it to `/workspace/source`, and open with `O_NOFOLLOW`.
- Read verdicts from state the wrapper owns, or only from the very last log line.
- Accept `/meta` once, and only for the commit the checkout resolved.

MEDIUM

CI-08 `ci_pipelines:edit` effectively grants read access to every CI secret

Location backend/api/services/ci/secrets.py:211-266; backend/api/services/ci/pipelines.py:263; backend/api/services/ci/logs.py:40-60; backend/api/services/ci/shared_pipelines.py:345

Confidence Confirmed

Who can exploit Users with `ci_pipelines:edit` (or `ci_builds:run` on a pipeline that already references secrets) who hold no `ci_secrets:*` permission.

Description

A stage may reference any global or same-service secret by name; `_secret_refs` only checks that the name exists. Values arrive as plain environment variables. Log masking replaces exact values only, so any encoding of the value (base64, sending it elsewhere) gets past it. Permissions are global, so every service's secrets are reachable, and attaching a shared pipeline adds the pipeline home's secrets to the service's builds.

Evidence

```
rows = CiSecret.query.filter(db.or_(CiSecret.service_id.in_(owners), CiSecret.scope == "global")).all()
env[ref.get("envVar") or name] = resolved[name]
```

Impact

The split between `ci_secrets:manage` and pipeline edit gives no confidentiality.

Recommendation

- Add per-secret scopes: allowed services or pipelines, and protected branches only.
- Require `ci_secrets:manage`, or a new `ci_secrets:use`, to add a `secretRefs` entry.
- Mask encoded forms of each secret.

LOW

CI-09 Merge-check pipelines run PR code automatically with no guard on Deploy or store-upload stages or on secrets

Location backend/api/services/ci/merge_checks/__init__.py:1123-1136; backend/api/services/ci/pipelines.py:1253-1260

Confidence Confirmed

Who can exploit An external Bitbucket user who can push a branch and open a PR, but only if a pipeline editor has put secrets or a Deploy or store-upload stage into the merge-check pipeline.

Description

A PR webhook builds the PR's source branch and commit with the merge-check pipeline. Nothing stops a `merge_check`-purpose pipeline from containing deploy or store_upload stages, cleanup commands, or arbitrary secret refs. `sourceBranch` and `commitSha` are truncated but not validated with `_valid_ref` / `_COMMIT_RE`.

Recommendation

Forbid those stages and any secret not on an allowlist in merge-check pipelines, and validate the ref and commit.

LOW

CI-10 Post-action webhooks are a blind SSRF from the backend pod

Location backend/api/services/ci/post_actions.py:1308-1355

Confidence Confirmed

Who can exploit A user with `ci_secrets:manage` together with `ci_pipelines:edit`.

Description

`_send_webhook` POSTs JSON from the backend to any http(s) URL stored in a CI secret. There is no block on private, link-local or cluster ranges, redirects are followed, and the HTTP status is shown back to the user.

Recommendation

Refuse private, loopback, link-local and cluster ranges after DNS resolution, disable redirects, and optionally allowlist hosts.

LOW CI-11 Code-scan "Send" and email post actions send arbitrary text to arbitrary external addresses

Location backend/api/routes/ci.py:1028-1058; backend/api/services/ci/code_scan_report.py:627-680; backend/api/services/ci/post_actions.py:243-253

Confidence Confirmed

Who can exploit Any user with `ci_artifacts:view` (in the default Viewer role) for Send, and `ci_pipelines:edit` for email post actions.

Description

The code-scan PDF (source findings) plus a free-text note, and pipeline-defined emails, go to arbitrary addresses through the organisation's SMTP. The addresses are only syntax-checked.

Impact

The organisation's mail server can be used as a phishing relay, and a Viewer can easily send source findings outside the organisation.

Recommendation

Restrict recipients to active users or allowed domains, require a manage-level permission, and rate-limit.

LOW CI-12 Any CI user can use the stored Bitbucket credentials against any repository

Location backend/api/routes/ci.py:224-238; backend/api/services/ci/catalog.py:907-955, :610-650

Confidence Confirmed

Who can exploit `ci_services:view` holders (Viewer) for listing; `ci_services:edit` holders for reading files and cloning.

Description

`POST /api/ci/source/revisions` lists branches, tags and commits of any Bitbucket repo using any enabled credential profile. A `ci_services:edit` user can bind any profile to any repo and then read or clone it. Credential profiles have no repository scope.

Recommendation

Scope credential profiles to a workspace or a repository allowlist.

LOW CI-13 Webhook secrets are accepted in the URL, and signed deliveries can be replayed

Location backend/api/routes/ci_webhooks.py:188-195; backend/api/routes/ci_merge_checks.py:305; backend/api/services/ci/webhook_triggers.py:908-926, :1040-1046

Confidence Confirmed

Who can exploit Anyone who sees ingress or proxy logs, or who captures a signed request.

Description

`?secret=` is accepted on both hooks, so the secret ends up in access logs. The HMAC carries no timestamp. De-duplication relies on a delivery header the caller chooses, plus a 10-minute window that applies only when a commit is present. A captured generic delivery without a commit can therefore be replayed indefinitely (up to 12 builds a minute), and each replay re-runs any Deploy stage.

Recommendation

Stop accepting secrets in the query string, add a signed timestamp, and reject old deliveries.

LOW CI-14 .dockerignore only excludes database files at the root, and the backend image is also the CI worker image

Location backend/.dockerignore; k8s/ci-backend-config.yaml (CI_WORKER_IMAGE)

Confidence Confirmed for the pattern behaviour. Not exploitable today, because release images are built in Bitbucket Pipelines from git, where `instance/` is ignored.

Who can exploit Any build stage (a pipeline editor can name this image as a stage image), but only if an image is ever built from a working copy.

Description

The `*.db` pattern only matches at the context root. backend/instance/ (SQLite databases and their `.bak` copies, tofu state, analysis artifacts) would be baked into kubesight-backend, which every build pod runs.

Recommendation

Add `instance/`, `**/*.db*`, `**/tofu/`, `*.log` and `tests/` to backend/.dockerignore.

6.4 Integrations, webhooks and outbound requests

Scope reviewed

Routes: ticketing.py, zoho.py, integrations.py, infra_connections.py (vSphere part), registries.py, mobile_apps.py, application_intelligence.py, alerts.py, alert_policies.py, alert_routing.py, topology_components.py (heartbeat and health-check URLs), ticket_agent.py (Telegram), the email-link routes in deployment_requests.py and change_bundles.py; Ticketing and deploy automation: zoho_client, zoho_sync_service, jira_client, jira_sync_service, jenkins_client, deploy_automation_service (webhook → auto-run → handoff); Registries and vSphere: registry_client, registry_service, vsphere_service; Alerts and email: alert_routing_service, alert_policy_service, alert_notifier, email_delivery, and the email paths of auth_service, deployment_request_service and change_bundle_service; Mobile: mobile_app_service, google_play_client, app_store_client, resign_executor, binary_signature; Application Intelligence: application_intelligence_service, _security, _bitbucket, application_analysis_jobs, application_analysis_local_docker, application_pull_request_jobs, application_checkout, application_worker, application_pull_request_worker, application_build_verifier, backend/Dockerfile.application-worker; Also: integrations_service, topology_component_service, Telegram press handling in ticket_agent/engine.py, default role grants in rbac_data.py.

HIGH

INT-01 Build-verifier code shares a writable HOME and workspace with the credentialed containers in Application Intelligence jobs

Location backend/api/services/application_pull_request_jobs.py:63-68, 75, 140-156, 191, 197; backend/api/services/application_analysis_jobs.py:197-199, 265, 282, 325, 342, 398; backend/api/application_build_verifier.py:30-48; backend/api/application_pull_request_worker.py:189-225

Confidence Confirmed. The code path was read end to end and the interpreter start-up behaviour was verified locally; it was not run on a cluster.

Who can exploit - An author of code in an analyzed repository, when that code is analyzed in Build Verified mode or turned into a pull request. - A user with `applications:manage` + `applications:analyze` (default Operator role), who can analyze a repository they control. When they request a pull request, they can choose any write-enabled credential profile, including admin-created ones shared with CI.

Description

The "credential-free" build verifier runs repository-defined build and test commands. In Kubernetes mode it is an init container in the same pod as the containers that hold credentials: `pr-publish` has the Bitbucket write token, and `analyzer` has the Hermes API token and the callback token. All of them share two writable emptyDir volumes: `/tmp`, which is also `HOME`, and `/workspace`, which includes `.git`. The credentialed containers then start Python with user-site loading enabled, and `pr-publish` runs git in the shared workspace, reading the shared HOME config and the repository's `.git` config and hooks. So files the untrusted build leaves behind can change what the trusted containers execute.

Evidence

```
volume_mounts = [{"name": "workspace", "mountPath": "/workspace"},
                 {"name": "tmp", "mountPath": "/tmp"}]
build_container = {..., "env": [{"name": "HOME", "value": "/tmp"}, ...], "volumeMounts": volume_mounts}
"initContainers": [pr_container("prepare"), build_container],
"containers": [pr_container("publish")],
```

Impact

Repository-controlled code can get the Bitbucket write credential or the Hermes token and push changes outside PR review. For Operators, that means reaching credentials only an admin should hold.

Recommendation

- Run the build verifier in its own pod or Job, with a copy of the source without `.git` and a private `/tmp` and `HOME`.
- In credentialed containers, run Python isolated (`-I` or `PYTHONNOUSERSITE=1`) with a private `HOME`.
- Push from a fresh clone or bundle, with hooks disabled and global and system git config ignored (`GIT_CONFIG_GLOBAL=/dev/null`, `GIT_CONFIG_NOSYSTEM=1`, `core.hooksPath` set to an empty directory).
- Allow only admin-approved credential profiles as PR write credentials.

MEDIUM INT-02 Changing an integration's URL keeps its stored secret, and the URL fields allow SSRF with partial response reflection

Location backend/api/services/zoho_sync_service.py:451-473, 549-562; backend/api/services/zoho_client.py:132-158; backend/api/services/jira_sync_service.py:162-185, 530-549; backend/api/services/deploy_automation_service.py:188-192, 313-319; backend/api/services/registry_service.py:121-144, 264-283; backend/api/services/vsphere_service.py:79-103

Confidence Confirmed

Who can exploit Users with `ticketing:manage` (Zoho, Jira, Jenkins), `registries:manage` or `vsphere:manage`. These are admin-only by default but can be granted to custom roles.

Description

The secrets are write-only in the API, and a blank secret on update keeps the stored value. But the base URL and token-endpoint fields accept any URL, with no host, scheme or address-range checks. After a URL change, Test and the scheduled sync send the decrypted credential to the new destination. The outbound clients use urllib defaults: they follow redirects (keeping the `Authorization` header across hosts) and accept `file://`. Test messages reflect status codes, socket errors and, for Zoho token errors, the full response body.

Evidence

```
for key, attr in (("apiBase","api_base"),("accountsBase","accounts_base"),("tokenEndpoint","token_endpoi
    nt"), ...):
    if key in payload and payload.get(key) is not None:
        setattr(row, attr, str(payload.get(key)).strip())
detail = payload.get("error") or payload.get("raw") or f"HTTP {status}"
```

Impact

A user who may configure an integration but not see its secret can obtain the credential, and can probe or partly read internal HTTP services from the backend pod.

Recommendation

- Clear the stored secret, and require it again, whenever the URL or host changes.
- Pin the Zoho OAuth endpoints to the Zoho data-centre hosts.
- Add one shared outbound-URL validator: https only; resolve the host and refuse loopback, link-local, metadata, unlisted private ranges and the cluster API.
- Disable redirects, or drop `Authorization` on a cross-host redirect.
- Stop echoing raw response bodies in Test results.

MEDIUM INT-03 The alert policy catalog returns webhook and Slack URLs to read-only users

Location backend/api/routes/alert_policies.py:21-24; backend/api/services/alert_policy_service.py:115-118, 139-153, 598-603

Confidence Confirmed

Who can exploit Any user with `alerts:view` (default Viewer role).

Description

Receiver management is admin-only, but `GET /api/alert-policies/catalog` returns every enabled receiver's `destination`, which for Slack and webhook receivers is the full URL. Slack incoming-webhook URLs and many generic webhook URLs carry their auth token in the path or query.

Evidence

```
def _receiver_destination(receiver):
    if receiver.receiver_type == "email":
        return receiver.email_address or ""
    return receiver.url or ""
```

Impact

Read-only users can get notification-channel credentials and post spoofed messages to ops channels.

Recommendation

Return only id, name and type from the catalog. Show URLs only on admin routes, masked to scheme and host.

MEDIUM INT-04 Component health checks let Operators make the backend reach any host or port

Location backend/api/routes/topology_components.py:29-37, 48-56, 67-74; backend/api/services/topology_component_service.py:115-128, 234-264

Confidence Confirmed

Who can exploit Users with `components:create` or `components:update` plus `components:check` (default Operator role).

Description

The HTTP check accepts any http(s) URL, follows redirects and returns the status code, latency or connection error. The TCP check connects to any host and port. Neither restricts internal, link-local or cluster-internal destinations, and the checks also run on the automatic health refresh.

Evidence

```
if not (url.startswith("http://") or url.startswith("https://")):
    return None, "Health check URL must start with http:// or https://."
with socket.create_connection((host, port), timeout=_TCP_TIMEOUT_SECONDS):
```

Impact

A non-admin role can map the internal network and reach internal endpoints from inside the KubeSight pod. No response bodies are returned.

Recommendation

Apply the INT-02 validator plus an admin-managed allowlist of CIDRs and hosts for health checks, and do not follow redirects.

MEDIUM INT-05 Security emails insert request-derived values into HTML without escaping

Location backend/api/services/auth_service.py:143-149, 267-290, 331, 343; backend/api/email_delivery.py:394-439, 442-480

Confidence Confirmed

Who can exploit Unauthenticated callers, because lockout emails are triggered by failed sign-ins against a known username.

Description

Three email builders interpolate values into the HTML body without escaping:

- `send_security_event_email`: IP address, headline, lines and full name;
- `send_login_notification_email`: user agent, IP address and full name;
- `send_temporary_password_email`: username and full name.

The IP comes from the raw `X-Forwarded-For` header (see AUTH-14), and the user agent is fully attacker-chosen.

Evidence

```
forwarded = request.headers.get("X-Forwarded-For", "")
if forwarded:
    return forwarded.split(",")[0].strip()
ip_html = f"<li>IP address: {ip_address}</li>" if ip_address else ""
```

Impact

Attacker-chosen markup reaches users and admins inside genuine KubeSight emails, which enables phishing from a trusted sender.

Recommendation

`html.escape()` every interpolated value in the three builders, as `_build_alert_html` already does, and take the client IP from `request.remote_addr` behind a correctly configured ProxyFix.

MEDIUM INT-06 Ticket webhooks trust the whole payload, have no replay protection, and can name any stored target

Location backend/api/routes/ticketing.py:856-895; backend/api/services/zoho_sync_service.py:1508-1516, 1562-1566, 1611-1619, 1644; backend/api/services/jira_sync_service.py:737-743; backend/api/services/deploy_automation_service.py:3114-3142

Confidence Confirmed

Who can exploit A webhook caller holding the provider's inbound secret, and, through the ticket's own fields, external ticket authors.

Description

- The webhook is authenticated by one static shared secret, which is also accepted as `?secret=` and so ends up in access logs.
- Tag, variable and value are taken from the payload as-is; the ticket is never re-read from Zoho or Jira.
- An explicit snapshot-id field skips the source-cluster scoping used by the normal name lookup.
- Auto-run is "once per ticket record", but the intake keeps only 10 records. Once a record is pruned, replaying the same delivery creates a new record and a new run.
- On auto-run clusters with no approval requirement, the change is applied directly.

Evidence

```
explicit_id = _extract(payload, "snapshot_id", "workload_id", "deployment_id", ...)
if explicit_id is not None:
    snapshot = ZohoDeploymentSnapshot.query.get(int(str(explicit_id).strip()))
INBOUND_TICKET_RETENTION = 10
```

Impact

Whoever holds the secret, or authors tickets, can deploy tags or change env vars on any recorded deployment on auto-run clusters without human review, and can replay old deliveries.

Recommendation

- Use HMAC-signed deliveries with a timestamp and nonce, or at least keep a delivery-id ledger that is never pruned, and drop the query-string secret.
- Remove the explicit-id path, or scope it to the provider's source cluster and namespaces.
- Re-fetch the ticket from the provider API before auto-run.
- Validate the tag format, and limit which env vars a ticket may change.

LOW

INT-07 The component webhook token is returned to view-only users

Location backend/api/services/topology_component_service.py:68-71, 155-159; backend/api/routes/topology_components.py:39-46, 78-90

Confidence Confirmed

Who can exploit Users with `components:view` (default Viewer role).

Description

`get_component` serializes with `include_webhook=True`, so `GET /api/topology-components/<id>` returns the heartbeat token, although the code comment says only users who can manage the component should see it. That token authorizes the unauthenticated heartbeat endpoint, which sets the component's status.

Impact

View-only users can falsify component health, hiding outages or raising false service alerts.

Recommendation

Return the token only on create, update or rotate, and only to users with `components:update`.

LOW

INT-08 Slack alert text inserts log content unescaped

Location backend/api/services/alert_routing_service.py:864-877

Confidence Confirmed

Who can exploit Anyone who can make a monitored workload log text that matches a log-alert pattern, for example an external user of the application.

Description

Log lines and pod and resource names are inserted into Slack mrkdwn inside a code fence without escaping. Content that closes the fence is then rendered as Slack formatting (mentions, links).

Impact

Spoofed links or mentions can appear in alerts posted by KubeSight's Slack integration.

Recommendation

Escape `&`, `<`, `>` and backticks in all interpolated fields, or send log lines as an attachment or a plain-text block.

LOW INT-09 The analysis revision is not validated before being passed to git

Location backend/api/services/application_intelligence_service.py:926-928; backend/api/application_checkout.py:113-126

Confidence Confirmed for the input path; impact is limited because HTTPS transport is used

Who can exploit Users with `applications:analyze` (default Operator role).

Description

`revision` is only stripped and then passed as a positional argument to `git fetch` with no `--` separator, so a value beginning with `-` is read as an option.

Impact

Unexpected git fetch behaviour in the checkout container; a defense-in-depth gap.

Recommendation

Validate the revision against a ref or SHA pattern that rejects a leading `-`, and put `--end-of-options` before it.

LOW INT-10 The mobile artifact workspace path can address any Jenkins path with the shared Jenkins credentials

Location backend/api/services/jenkins_client.py:443-447; backend/api/services/mobile_app_service.py:515-520, 1036-1040

Confidence Confirmed

Who can exploit Users with `mobile_apps:manage` (admin by default).

Description

`workspace_file_url` URL-encodes each path segment but keeps `..` segments, so the configured workspace path can point outside the build workspace. The response is fetched with the deploy-automation Jenkins credentials and stored as a downloadable build for `mobile_apps:view` holders.

Impact

Jenkins content outside the job workspace could become readable by mobile viewers.

Recommendation

Reject `.` and `..` segments and absolute paths in the workspace path, and check that the final URL stays under the build's workspace prefix.

LOW INT-11 Registry check-image puts the repository and tag into the request path unvalidated

Location backend/api/routes/registries.py:91-99; backend/api/services/registry_client.py:262-264

Confidence Confirmed

Who can exploit Users with `registries:view`.

Description

The parsed repository and reference are concatenated into `/v2/<repository>/manifests/<reference>` with no character validation. That allows path and query manipulation of the HEAD requests made with the stored registry credentials.

Impact

Minor: HEAD requests to unintended paths on a configured registry, made with its credentials.

Recommendation

Validate the repository against the distribution-spec name grammar and the reference against the tag or digest grammar before building the URL.

LOW INT-12 Inbound ticket records are keyed by ticket id across providers

Location backend/api/services/zoho_sync_service.py:1562-1566; backend/api/models.py:2175-2179

Confidence Confirmed

Who can exploit A holder of one provider's webhook secret.

Description

The Zoho intake looks up an existing record by `ticket_id` alone and does not set `provider` on an existing row. A delivery can therefore overwrite another provider's ticket record, and later outcome write-backs go to the original provider's ticket.

Impact

One provider's webhook can change another provider's ticket records and status updates.

Recommendation

Key records by `(provider, ticket_id)`, with a unique constraint on that pair.

INFO

INT-13 The SMTP test is reachable with alerts:manage, while the equivalent route is admin-only

Location backend/api/routes/alerts.py:129-136; backend/api/alert_notifier.py:108-109

Confidence Confirmed

Who can exploit Users with `alerts:manage` (default Operator role).

Description

This route calls the same `send_smtp_test` that `/api/alert-routing/smtp/test` protects with `require_admin`. It sends a fixed test email to any address and updates the SMTP last-test status.

Impact

The gating is inconsistent, allowing limited mail relay of fixed content.

Recommendation

Gate it with `require_admin`, or restrict the recipient to the caller's own email.

6.5 MCP server and Hermes agent

Scope reviewed

backend/api/mcp/** (__init__, access.py, protocol.py, tools/registry.py, tools/common.py and all 10 tool modules). All 105 tools were enumerated from the code: 34 write, 71 read. Every write tool and every cluster-touching tool was traced into its service for permission, namespace access, the per-cluster check and the approval gate; routes mcp.py, ticket_agent.py, api_tokens.py; auth_utils.py; decorators.require_auth; services/ticket_agent/* (engine, validator, schema, catalog, hermes, settings, telegram, vm_requests); relevant parts of deploy_automation_service, services/ci/pipelines.py, services/ci/deploy_stage.py, resource_actions_service.exec_in_pod, rbac_data, seed.py; connect-hermes-to-kubesight.sh, KUBESIGHT-MCP.md, docker-compose.hermes-local.yml, hermes-local.runtime.env, Dockerfile.hermes-local, Dockerfile.hermes-production; k8s/kubesight-hermes-integration.yaml, k8s/hermes-mcp-egress.yaml, k8s/hermes-application-intelligence.yaml; The gitignored local hermes/ snapshot (key names only, values not printed), git history for tokens, agent/kubesight-agent.py.

HIGH

MCP-01 A live-format KubeSight API token is committed and pushed to both remotes (previously owner-accepted)

Location connect-hermes-to-kubesight.sh:18

Confidence Confirmed that it is present and pushed. Whether the token is still valid could not be checked, because the DB was off-limits.

Who can exploit Anyone with read access to the GitHub or Bitbucket repository, or any clone of it. No KubeSight account is needed.

Description

The token has the exact format minted at api_tokens.py:83 (`ksa_` + 40 hex). The same literal is also used as the "did you edit me?" guard on line 24, which shows the real token replaced the placeholder in both places (and makes the script always exit). ksa_ tokens have no scopes, so this is a full REST and MCP credential for its owner.

Evidence

```
KUBESIGHT_TOKEN="ksa_4a28***"   # value masked in this report
if [ "$KUBESIGHT_TOKEN" = "ksa_4a28***" ]; then
```

Impact

An attacker can do everything the token owner's role allows: deploys, Helm, exec and CI. If the owner is an admin, that means full control of every managed cluster.

Recommendation

- Revoke the token with prefix `ksa_4a28` now.
- Replace it in the script with `${KUBESIGHT_TOKEN:?}` and a fixed placeholder guard.
- Purge it from history on both remotes (git filter-repo).
- Add a secret scan for `ksa_[0-9a-f]{40}` to CI.

HIGH

MCP-03 Ticket tools are not bound to the ticket Hermes was given, so a prompt-injected ticket can read and act on every other ticket

Location	backend/api/services/ticket_agent/engine.py:459 (`_ticket`), :469 (`_current_task`)
Confidence	Confirmed that there is no server-side binding. Exploitation is Likely, because it depends on the model following injected text.
Who can exploit	Anyone who can create a ticket, add a comment, or reply by email in the connected Zoho Desk department or Jira project (often external requesters).

Description

Every `kubesight_ticket_*` tool accepts any `ticketRecordId`. `_ticket()` only checks that the record exists, and `_current_task()` opens a RUNNING task on the spot for any ticket. Even with the documented `hermes-agent` role (`ticketing:view` + `ticketing:agent`), injected text can lead Hermes to:

- list every inbound ticket, including `variableValue` and other tickets' troubleshooting diagnoses (services/zoho_sync_service.py:1746-1765);
- fetch the live description and all Desk comments, including internal notes, because there is no `isPublic` filter (zoho_sync_service.py:1913-1918);
- post that content on the attacker's own ticket, where comments are public by default (settings.py:52-54);
- comment on, close or park other requesters' tickets, or run `kubesight_ticket_execute` against another ticket record.

The only barrier is a prompt sentence saying that ticket text is untrusted data (hermes.py:44).

Evidence

```
def _ticket(record_id):  ticket = db.session.get(ZohoInboundTicket, int(record_id)); ... return ticket
def _current_task(ticket, user=None):
    task = ...status.in_((RUNNING, PENDING))...first()
    if task is not None: return task
    return _new_task(ticket, "handle", status=RUNNING, requested_by=... or "hermes")
```

Impact

Data can leak from one requester to another, the support identity can be used to tamper with other tickets, and changes can run on targets taken from other tickets.

Recommendation

- Give each Hermes task its own random token and require it in every ticket write tool. Refuse when the `ticketRecordId` does not match that task, or when the task is not RUNNING.
- For the `hermes-agent` account, limit `ticket_get` / `tickets_list` to the current ticket.
- Feed Hermes only public comments, and default `public_comments` to False.

HIGH

MCP-04 The ticket agent's "use only ticket tools" rule exists only in the prompt, and the design runs it on the operators' shared Hermes and token

Location	backend/api/services/ticket_agent/hermes.py:3 (also :13, :182)
Confidence	Likely. The code path is confirmed; the privilege of the token actually deployed cannot be seen from the repo.
Who can exploit	A ticket author (as in MCP-03), whenever the Hermes at `TICKET_AGENT_HERMES_URL` holds a token broader than the `hermes-agent` role.

Description

- The docstring says the ticket agent is the operators' own Hermes, carrying the kubesight skill and the KubeSight MCP connection.
- The system prompt tells Hermes to troubleshoot with `inventory_list`, `pod_logs`, `resource_get`, `namespace_events` and `alerts_list`. The fixed `hermes-agent` role (rbac_data.py:521-527) has none of the matching view permissions, so troubleshooting only works if operators give Hermes a broader user's token.

- The instruction "never call deploy / restart / scale / helm / build tools" (hermes.py:182) is prompt text only. There is no server-side tool allowlist per caller: `McpAccessSettings` is global.
- The prompt also tells Hermes to quote log lines into comments, which are public by default.

Impact

With an operator or admin token behind Hermes, a crafted ticket can lead to direct deploy, Helm, scale or exec calls (exec is ungated, K8S-05), and to log lines or environment values being posted to an external requester.

Recommendation

- Make the ticket agent refuse to run unless its Hermes MCP token resolves to `hermes-agent`.
- Enforce a server-side allowlist for that identity in routes/mcp.py: the ticket tools plus a named set of read tools, scoped to catalog namespaces.
- Redact environment and secret values from anything that can reach a public comment.
- Fix the stale docstring at hermes.py:13.

MEDIUM

MCP-05 Any ticket author can trigger unreviewed production changes; the only gate is the model's self-reported confidence

Location backend/api/services/ticket_agent/validator.py:93

Confidence Confirmed

Who can exploit Anyone who can file a ticket. No requester identity or allowlist is checked anywhere in ticket_agent/.

Description

A run goes straight to execution with no human when the target is in the catalog, Hermes claims High confidence (copied from the tool argument), and the dropdowns are empty or agree, which is the case for most free-text tickets. One ticket can carry up to 20 changes. Each can deploy any tag matching `TAG_RE` (built first if absent), restart a workload, or set an existing literal env var to any value up to 2000 characters, for example to repoint a database or API URL. A cluster approval gate still applies where the cluster has one.

Evidence

```
if _RANK.get(request["confidence"], 0) < _RANK.get(min_confidence, 3): batch.reasons.append(...)
...
if batch.reasons: batch.route = "approval"
return batch   # otherwise execute, no human
```

Impact

Outsiders can roll production back to old or vulnerable tags, cause restarts, or redirect configuration.

Recommendation

Add a requester allowlist (contact IDs, domains, Jira groups) enforced server-side in `execute` and in `start_run(origin="agent")`. Always require approval for `set_env_var` and for production environments.

MEDIUM

MCP-06 Per-cluster "Read only" and "Off" MCP rules are bypassed through CI tools that deploy indirectly

Location backend/api/mcp/tools/ci.py:1858

Confidence Confirmed

Who can exploit A token with `ci_builds:run` / `ci_builds:retry` on a service whose pipeline has a Deploy stage, or with `ci_pipelines:edit` + `apps:deploy` to add a new target in a closed cluster.

Description

`check_cluster` is only reached through `resolve_cluster`, `keep_allowed` and a few explicit calls. The CI write tools (`build_run`, `build_retry`, `pipeline_save`, stage add/update, `shared_pipeline_attach`, `service_create`) are not `cluster_scoped`. The Deploy stage runs in the engine thread, where the MCP context variable is unset, so the check does nothing there. `_stamp_deploy_authority` (services/ci/pipelines.py:837) checks `apps:deploy` and namespace access but not the MCP rule. The approval gate still applies.

Evidence

```
@tool("kubesight_build_run", permission="ci_builds:run", ..., write=True, schema={...})  # no cluster_sc
    oped
def _build_run(arguments, *, user=None): data = engine_service.trigger_build(row, ..., actor=user, ...)
```

Impact

An admin's "agents may not change this cluster" switch does not stop an agent from deploying to that cluster.

Recommendation

Mark the CI write tools `cluster_scoped`. In run and retry, resolve every Deploy target of the pipeline (including linked ones) and call `check_cluster` for each, and do the same in the save, attach and create paths for any targets they add or change.

MEDIUM

MCP-08 The remote build agent runs pipeline commands on its host; the pipeline can switch off its container boundary, and artifact paths are not confined

Location agent/kubesight-agent.py:901 (also :575 and :758)

Confidence Confirmed

Who can exploit Anyone who can edit a pipeline routed to an agent runner (`ci_pipelines:edit`, in the UI or through the MCP pipeline tools).

Description

The container mode comes from the stage's own env, so a pipeline can set `KUBESIGHT_CONTAINER=never` (line 575). Host-mode stages inherit `**os.environ` (line 758), which includes `KUBESIGHT_AGENT_TOKEN` when the agent was started with that env var. The artifact glob `os.path.join(source, workdir, path)` accepts absolute or `../` paths, and the server does not validate artifact paths (services/ci/pipelines.py:288-303). So even a containerised stage can have the agent upload host files such as `~/.ssh/id_rsa`.

Evidence

```
pattern = os.path.join(source, spec.get("workdir") or "", spec.get("path") or "")
matches = [p for p in glob.glob(pattern, recursive=True) if os.path.isfile(p)]
```

Impact

Code execution and file theft on build hosts that KubeSight otherwise cannot reach (Mac minis with signing identities, licensed toolchains), including theft of the agent token.

Recommendation

- Add an agent-side `--require-container` flag that a stage cannot override.
- Strip the agent token from the stage environment.
- Confine artifact matches to `realpath(source)` and reject absolute or `..` paths, on both the server and the agent.

LOW

MCP-09 JSON-RPC batches are unbounded, and the MCP audit row hides repeated calls

Location backend/api/mcp/protocol.py:194; backend/api/routes/mcp.py:131

Confidence Confirmed

Who can exploit Any authenticated token, including a viewer's.

Description

A batch array is run synchronously with no cap, and there is no body-size limit. The audit row's `count` is the number of distinct tool names, so thousands of calls in one POST show up as a single "1 tool" row (the services still write their own rows).

Impact

Backend workers can be exhausted, and the audit trail under-reports the activity.

Recommendation

Cap batch length (for example at 20) and set `MAX_CONTENT_LENGTH`. Count every `tools/call` in the audit row, and add per-token rate limiting on /api/mcp.

LOW MCP-10 Telegram approvals trust a changeable username, and an empty approver list lets anyone in the chat approve

Location backend/api/services/ticket_agent/engine.py:1221 (also :1022, :1255)

Confidence Confirmed

Who can exploit Members of the approval chat, or anyone who can take over an approver's @username in a setup that uses an @channel.

Description

`_allowed()` returns True when the approver list is empty. List entries match either the numeric id or the username, and usernames can be changed and re-claimed. There is no KubeSight identity or permission check on Telegram approvers. The approval message includes the raw multi-line ticket subject and Hermes's summary of the request, which can render fake "Will:" lines above the real, server-generated one.

Evidence

```
def _allowed(sender, allow):
    if not allow: return True
    ids = {str(sender.get("id") or "").casefold(), str(sender.get("username") or "").casefold()}
```

Recommendation

Require a non-empty list of numeric Telegram user IDs, mapped to KubeSight users who hold `ticketing:manage`. Collapse newlines in the subject and summary, and put the server-generated "Will:" block first.

LOW MCP-11 API tokens have no scopes and no default expiry, so every MCP token is a full credential for its user

Location backend/api/routes/api_tokens.py:76

Confidence Confirmed

Who can exploit Not directly exploitable on its own; it widens the impact of MCP-01, MCP-04 and PLAT-06.

Description

Tokens are SHA-256 hashed and are checked for revocation, expiry and inactive users. But `expires_at` is optional with no maximum, and there is no scope, so a token handed to an agent "for MCP" also works on every REST route the user can call.

Recommendation

Add scopes or an audience (MCP-only, plus a tool allowlist) and enforce them in `get_current_user` and the MCP route. Default new tokens to an expiry of 90 days or less.

LOW MCP-12 Unknown-tool errors list every registered tool, including ones the admin switched off

Location backend/api/mcp/tools/registry.py:232

Confidence Confirmed

Who can exploit Any authenticated token.

Description

An unknown tool name is answered with `"Available: " + ", ".join(sorted(_REGISTRY))`. That is the whole registry, including tools the admin switched off, which contradicts the rule that a switched-off tool is absent.

Recommendation

List only `access.filter_definitions(definitions(user=user))`.

6.6 Cluster Builder, SSH and VM provisioning

Scope reviewed

master HEAD dad6d8b6. New VM code since 2026-09-26: OpenTofu 54c75c99, day-two growth + etcd 2428b8fe, VMs-only 33bbe0d9, Hermes VM requests fa3b4d9a; routes cluster_builds.py, cluster_provisioning.py, infra_connections.py; services cluster_build/{service, executor, preflight, storage, workloads, profiles, scrub, lb, kubeadm, os_adapters/base}; cluster_build/provisioning/{service, jobs, tofu_config, tofu_runner, state_store, inventory, ip_pool, templates, vm_watch}; ssh/{transport, hostkeys}, ssh_profile_service, vsphere_client, vsphere_service; ticket_agent/vm_requests (+ engine/settings approval paths), MCP kubesight_ticket_request_vms; rbac_data, models_cluster_build, the OpenTofu HTTP state backend, Dockerfile, k8s_entrypoint.sh, nginx.conf, the untracked test file.

MEDIUM

PROV-01 Workload copy reads source-cluster Secret values without `secrets:reveal`

Location backend/api/services/cluster_build/workloads.py:349 (authorize_selection), :515 (Secret data carried verbatim); backend/api/services/cluster_build/executor.py:1918 (export at build time)

Confidence Confirmed

Who can exploit An authenticated user with `cluster_builds:create`, `cluster_builds:execute` and view access to a namespace on a source cluster. To read the values back they also need `cluster_builds:kubeconfig` or control of the new cluster.

Description

"Bring workloads from an existing cluster" copies every ConfigMap and Secret that a selected pod references. The only gate is `authorize_selection`, which calls `can_access_cluster`/`can_access_namespace` and never checks `secrets:reveal`. Secret `data` is read with the source cluster's admin kubeconfig and written into the target cluster. Everywhere else in the app (REST, MCP, Helm), Secret values sit behind `secrets:reveal`. With `cluster_builds:kubeconfig` on the built cluster, the user can then read the imported Secrets.

Evidence

```
# authorize_selection — the whole gate (no secrets:reveal):
_check_access(user, source)
for namespace in sorted({item["namespace"] ...}):
    _check_access(user, source, namespace)
# _project_object keeps Secret values verbatim:
for key in ("data", "stringData", "binaryData", "type", "rules", "subjects", "roleRef"):
    if key in doc: out[key] = doc[key]
```

Impact

Every Secret value in a namespace the user can view can be exfiltrated, which bypasses the `secrets:reveal` boundary set by the previous hardening pass.

Recommendation

Require `secrets:reveal`, in addition to namespace access, before Secret data is exported or copied. Otherwise strip Secret values when the executing user lacks it, as resource_actions_service and helm_service already do.

MEDIUM

PROV-02 vCenter "Skip TLS verify" disables certificate validation for the provisioning account

Location backend/api/services/vsphere_client.py:47 (_ssl_context, CERT_NONE); backend/api/services/cluster_build/provisioning/inventory.py:236 (CERT_NONE for pyVmomi); backend/api/services/cluster_build/provisioning/jobs.py:512 (VSPHERE_ALLOW_UNVERIFIED_SSL passed to OpenTofu)

Confidence Confirmed

Who can exploit An attacker able to intercept traffic between KubeSight and vCenter, once an admin turns on skipTlsVerify for the connection.

Description

With `skip_tls_verify` on, all three vCenter clients (urllib REST, pyVmomi SmartConnect and the OpenTofu vSphere provider) accept any certificate and skip the hostname check. The provisioning account's credentials are then sent to whatever answers at that address. That account can clone and delete VMs, which makes it a valuable credential to steal.

Evidence

```
if cfg.skip_tls_verify:
    ctx = ssl.create_default_context(); ctx.check_hostname = False; ctx.verify_mode = ssl.CERT_NONE
# jobs.py: "VSPHERE_ALLOW_UNVERIFIED_SSL": "true" if cfg.skip_tls_verify else "false"
```

Impact

An interceptor captures the vCenter provisioning credentials and gains full VM create and delete rights in the datacenter.

Recommendation

A caPem option already exists. Show a clear warning and write an audit entry when skipTlsVerify is turned on, and consider refusing it when APP_ENV=production unless explicitly overridden. This is an admin-controlled defense-in-depth gap, not a code bug.

LOW

PROV-03 Default SSH host-key policy trusts the first key it sees (first-connect interception accepted silently)

Location backend/api/services/ssh_profile_service.py (profile default host_key_policy="tofu"); backend/api/services/ssh/hostkeys.py:54 (tofu records an unknown key); backend/api/services/ssh/transport.py (a changed key is refused)

Confidence Confirmed

Who can exploit An attacker who can intercept the first SSH connection from KubeSight to a node or bastion that uses the tofu policy.

Description

The tofu (trust on first use) policy records and trusts an unknown key on the first connection, with no out-of-band check. A key that changes later is always refused. For VMs that KubeSight creates the risk is lower, because the first connection is to a genuinely fresh VM and stale tofu keys for reused addresses are deleted before connecting (jobs.py:1363). Hosts and bastions that operators register by hand are more exposed. The stricter strict and pinned policies exist and are enforced.

Evidence

```
if policy == "tofu":
    db.session.add(SshHostKey(..., source="tofu")); db.session.commit()
    return True, "recorded on first use (tofu)"
```

Impact

If the first connection is intercepted, the session runs `sudo sh -s` as root on the node, so the attacker gets root on that machine.

Recommendation

Make pinned or strict the default for credentials and profiles used on existing hosts, and offer scan-then-pin before the first real connection. The current behaviour is documented; this is a hardening item.

LOW

PROV-04 The self-approval guard for destroys and VM requests is skipped when the request has no stored requester

Location backend/api/services/cluster_build/provisioning/service.py:519 (approve_destroy); backend/api/services/ticket_agent/vm_requests.py (check_approver)

Confidence Confirmed

Who can exploit Someone who can both request and approve, and only when the request was created with `requested_by_user_id = None`.

Description

The guard is `if user is not None and job.requested_by_user_id and job.requested_by_user_id == user.id`. If the request has no user id (future automation, a tokenless path, or a Hermes VM request with no requester), the "someone else must approve" rule is skipped without notice. Current UI routes always pass a real user, so the window is narrow, but the control fails open.

Evidence

```
if user is not None and job.requested_by_user_id and job.requested_by_user_id == user.id:
    raise PermissionError("You asked ... so someone else has to approve it.")
```

Impact

The two-person rule for VM destroy and create can be defeated in the (currently unlikely) case where the requester is not recorded.

Recommendation

Fail closed. When `requested_by_user_id` is None, require a different approving actor or refuse the approval; never treat a missing id as "anyone may approve".

LOW

PROV-05 Authenticated host and port probe through the SSH test and host-key scan endpoints

Location backend/api/routes/infra_connections.py:250 (test_ssh_profile), :282 (scan_ssh_host_key); backend/api/services/ssh_profile_service.py:325 (test_profile); backend/api/services/ssh/hostkeys.py scan_host

Confidence Confirmed

Who can exploit An authenticated user with `ssh_credentials:manage`.

Description

Both endpoints connect from the backend to any host and port the caller supplies. The scan does only the SSH key exchange; the test does a full login and runs id and uname. Their different error messages ("unreachable", "scan failed", authentication error) tell the caller what the backend can reach on its network. The permission is already marked as dangerous, so exposure is limited, but the endpoints still give a pivot from the backend pod.

Evidence

`socket.create_connection((host, port), ...)` in scan_host; `transport.run(target, "id -un && uname -sr")` in test_profile.

Impact

Internal network and port enumeration, plus minor information disclosure.

Recommendation

Acceptable for an infrastructure-admin permission. Optionally restrict targets to an allowlist or CIDR range and rate-limit the endpoints.

6.7 Approvals, promotions and governed delivery

Scope reviewed

Routes: approvals.py, deployment_requests.py, change_bundles.py (approval side), promotions.py, deployment_forms.py, inventory.py (wizard templates, actions), service_blueprints.py, application_services.py, clients.py, upgrades.py (approval side), ticket_agent.py (approve/reject), ci.py (stage approve/reject), cluster_provisioning.py (destroy approve); Services: approvals_service, deployment_request_service, change_bundle_service, change_bundle_executor, promotion_service, promotion_releases, promotion_timetable, deployment_form_service, deployment_form_parser, deployment_form_excel, inventory_actions_service, blueprint_deploy_service, blueprint_picker_service, ci/approval_stage, ci/deploy_stage, ci/deploy_templates, ticket_agent/engine, ticket_agent/vm_requests; Supporting: deployment_service.analyze_resources and validate_yaml, rbac_data.py, auth_utils.py, k8s/ingress.yaml, frontend/src/authStorage.js; Commits since 2026-09-26: dad6d8b6, 3a12deda, fe8c2222, fa3b4d9a, 96852449, c07610a3, c7f3103c, caa52297; Runtime check: K8S-01, 02, 03, 05 and 07 were also confirmed against an in-memory SQLite app in mock Kubernetes mode.

HIGH

APR-02 The email approval result page renders the cluster name unescaped (stored XSS)

Location backend/api/routes/deployment_requests.py:160, :178-182; backend/api/services/deployment_request_service.py:707

Confidence Confirmed

Who can exploit `deployment_requests:request`, which viewer, operator and cluster_admin have by default. The victim is whoever opens the approval link, usually an admin.

Description

`create_request` stores the `clusterName` from the request body as given; only `clusterId` is checked by `@require_cluster_access`. `_handle_email_action` inserts that value into an HTML response without escaping and sends no Content-Security-Policy. The page is served from the same origin as the SPA, which keeps the session token in localStorage.

Evidence

```
cluster = result.get("clusterName") or result.get("clusterId")
f"(cluster <strong>{cluster}</strong>), {detail} ..."
```

Impact

Script runs in the approver's session on the KubeSight origin, so a viewer can take over an admin's session.

Recommendation

- Escape every value inserted into `_action_result_page`, in both deployment_requests.py and change_bundles.py.
- Resolve the cluster name on the server from the cluster store instead of trusting the client.
- Add a restrictive CSP to these pages.

MEDIUM

APR-04 Email approval links change state on GET, are reusable, and are bound to a mailbox rather than an account

Location	backend/api/routes/deployment_requests.py:185-192; backend/api/routes/change_bundles.py:222-229; backend/api/services/deployment_request_service.py:56-94; backend/api/services/change_bundle_service.py:119-150
Confidence	Confirmed for the code. Likely for mail-scanner prefetch, which depends on the mail gateway.
Who can exploit	Anyone who receives the link: shared or distribution-list mailboxes, forwarded mail, URL-scanning gateways.

Description

A GET with a JWT in the query string records a vote, with no confirmation step and no single-use marker. The token carries an email address, not a user account. The self-approval check compares only against the requester's account email, so a requester who belongs to a distribution list in the approver pool also gets a working link. This applies to both deployment requests and change bundles.

Impact

Votes can be cast by automated link scanners or by people outside the intended approver set.

Recommendation

- Make GET show a confirmation page only, and record the vote with a POST.
- Make tokens single-use (store a nonce) and shorten their lifetime.
- Allow only pool members that map to user accounts, and exclude the requester by user id.

MEDIUM

APR-05 Approval rule changes, individual votes and some status changes are not audited

Location	backend/api/routes/deployment_requests.py:46-60; backend/api/services/deployment_request_service.py:454-536; backend/api/services/promotion_timetable.py:279; backend/api/services/ci/deploy_stage.py:498-527
Confidence	Confirmed
Who can exploit	`deployment_requests:manage` (admin by default) for rule changes; `apps:deploy` for timetable exclusions.

Description

- `PUT /api/deployment-requests/recipients`, which the Rules tab uses, changes per-cluster approval counts, the global count and the approver pool, and writes no audit record.
- Individual votes are stored only in the vote tables.
- Email-link finalizations are audited with no actor.
- `set_excluded` is not audited, and withdrawing a CI bundle changes its status directly with no audit.

Impact

An approval gate can be lowered and restored with no trace.

Recommendation

- Audit every recipients or settings write, with before and after values.
- Audit each vote with voter, channel and time.
- Audit timetable exclusions and bundle withdrawals.

MEDIUM APR-06 Who may approve is not tied to the approver pool or to cluster access, and the quorum is silently reduced

Location backend/api/routes/change_bundles.py:145-152; backend/api/routes/deployment_requests.py:97-104; backend/api/services/deployment_request_service.py:532, :743; backend/api/services/change_bundle_service.py:1062

Confidence Confirmed

Who can exploit `change_bundles:manage` or `deployment_requests:manage` (admin by default; custom roles may grant them).

Description

In-app votes are accepted from any holder of the manage permission, whether or not they are in the configured approver groups or can access the clusters involved. The required count is capped at the pool size after the requester is removed, so a cluster set to 2 approvals can need only 1, and an empty pool needs 1 from any manager.

Impact

The Rules tab's approver groups and per-cluster counts do not actually constrain who approves; a manager scoped to a dev cluster can approve production changes.

Recommendation

- Require the voter to be in the snapshotted approver pool (by user id) and to have access to every cluster in the item.
- Refuse to submit when the pool cannot meet the configured count, instead of lowering it.

MEDIUM APR-07 A deployment request approval has no time limit and no content binding

Location backend/api/services/deployment_request_service.py:716-727, :1014-1051

Confidence Confirmed (the runtime check accepted a 10-year window)

Who can exploit `deployment_requests:request`, once one approval is obtained.

Description

The requester sets the window end with no maximum. One approval then permits every change to the whole cluster until that end, and the permission also extends to CI Deploy stages and automation the requester has authorized.

Impact

A single approval can grant long-lived, cluster-wide deploy rights.

Recommendation

Cap the window length (configurable), optionally scope the approval to namespaces, and show the window length prominently to approvers.

MEDIUM APR-08 CI Deploy stages render inventory templates at deploy time, and any template editor can change them

Location backend/api/services/ci/deploy_stage.py:952-965; backend/api/services/ci/deploy_templates.py:50-58; backend/api/routes/inventory.py:442-449, :512-523

Confidence Likely. How far a template can widen a pod spec depends on the template schema.

Who can exploit `inventory:update` (cluster_admin by default).

Description

When a target deployment is missing, the Deploy stage renders from the template's current content and applies it as the stage's authorizing user. Templates have no owner check, so an editor can change a template that another team's pipeline uses to deploy into namespaces the editor cannot access.

Impact

Deployment content can change after the target was authorized.

Recommendation

Pin a template version or content hash when the stage target is saved, and refuse or re-authorize on mismatch. Restrict template edits to owners or admins.

LOW APR-10 The approval link base URL falls back to the request Host header

Location backend/api/services/deployment_request_service.py:213-223

Confidence Likely. The ingress host rule mitigates this in the shipped deployment.

Who can exploit `deployment_requests:request` or `change_bundles:create`, only where `PUBLIC_BASE_URL` is unset and the Host header is not enforced.

Description

When no public URL is configured, links in approval emails are built from `request.host_url`. No manifest in the repo sets `PUBLIC_BASE_URL`.

Impact

Approval links could point to a host other than KubeSight.

Recommendation

Require `PUBLIC_BASE_URL` in production and leave links out when it is missing, or enforce a trusted-hosts list.

LOW APR-11 The Excel form import has no upload size limit

Location backend/api/routes/deployment_forms.py:56-69; backend/api/services/deployment_form_parser.py:43

Confidence Confirmed

Who can exploit `apps:dryrun` (operator, cluster_admin).

Description

No `MAX_CONTENT_LENGTH` is set, the ingress allows 512 MB bodies, and openpyxl loads the whole workbook into memory. XXE is mitigated, because defusedxml is installed and openpyxl uses it.

Impact

A large or highly compressed workbook can exhaust backend memory.

Recommendation

Cap the upload size for this route, check the uncompressed zip size before loading, and use `read_only=True` with bounded reads.

LOW APR-12 Any deployer can change any environment's scheduled departures

Location backend/api/routes/promotions.py:148-173, :96-125; backend/api/services/promotion_timetable.py:262-298

Confidence Confirmed

Who can exploit `apps:deploy`.

Description

Hold, skip, exclude and early promotion (`slot`) of a departure check no namespace or environment access, and exclusions are not audited.

Impact

Users can disrupt other teams' scheduled releases.

Recommendation

Require access to the environment's bindings, or `promotions:manage`, and audit exclusions.

LOW APR-13 Pipeline editors control self-approval on the CI Approval stage, and triggered builds record no starter

Location backend/api/services/ci/approval_stage.py:74-91, :192-199

Confidence Confirmed

Who can exploit `ci_pipelines:edit` (cluster_admin), or a commit author on webhook- or cron-triggered builds.

Description

`allowSelfApproval` and the approver list are part of the pipeline being gated. Builds without `requested_by` skip the self-approval check.

Impact

A pipeline's own editors can weaken its approval gate.

Recommendation

Require a separate permission to change `allowSelfApproval` and the approvers. For triggered builds, attribute the build to the commit author or trigger owner and apply the self-approval check.

LOW

APR-14 Approval group endpoints expose the user directory and can edit alert-routing groups

Location backend/api/services/approvals_service.py:817-832, :858-935; backend/api/routes/approvals.py:60-103

Confidence Confirmed

Who can exploit `deployment_requests:view` (operator, cluster_admin) to read the user directory; `deployment_requests:manage` to edit or delete groups.

Description

`list_groups` returns the names and emails of every active user. Group save and delete act on any `AlertRoutingReceiverGroup`, including groups that alert policies use.

Impact

The user directory is disclosed, and alert routing can be changed under a different permission.

Recommendation

Return the user options only to users-management or manage roles. Restrict edits to groups marked as approver groups, or also require the alert-routing permission.

6.8 Frontend, infrastructure, secrets and dependencies

Scope reviewed

Frontend.; Covered frontend/src, index.html, vite configs, nginx.conf, the Dockerfile, package.json and package-lock.json; Searched for every XSS sink: dangerouslySetInnerHTML, innerHTML, document.write, insertAdjacentHTML, eval / new Function, srcdoc, postMessage, iframes and text/html blobs; Checked each href/src built from API data, where the JWT is stored, the hash-routing open-redirect guard, VITE_* use, the built dist/ bundle and the config.js loader; **Infrastructure.**; Reviewed all k8s/*.yaml, k8s/ci-cache.sh, custom-resources.yaml, the areeba-txm chart and the three docker-compose files; Read every Dockerfile, plus backend/k8s_entrypoint.sh, backend/app.py, backend/.dockerignore and bitbucket-pipelines.yml; **Secrets hygiene.**; Ran git grep over the working tree; Listed every file ever added (46,937 paths) and ran a pickaxe search over history; Checked which local and remote refs contain each secret-bearing commit, which logs are tracked, and .gitignore coverage; **Dependencies.**; Ran `npm audit --package-lock-only` on frontend/ (reads the lockfile only, installs nothing); pip-audit was not available, so Python advisories come from the auditor's knowledge as of mid-2026.

HIGH

PLAT-02 A live-format Telegram bot token and a second ksa_ token are in master history, pushed to GitHub and Bitbucket

Location k8s/hermes-secret.yaml. Real values were added in commit 12048785 (2026-06-19); the file was deleted in bb8a2519 (2026-06-25).

Confidence Confirmed that the tokens are present and pushed. Whether they still work cannot be checked from the repo.

Who can exploit Anyone with read access to the GitHub repository (12048785 is an ancestor of origin/master) or to the Bitbucket repository (it is in main / bitbucket-main), and anyone holding a clone.

Description

This is a different ksa_ token from the one in MCP-01, and it was not reported in the earlier audit. The Telegram token in history differs from the one in the current local Hermes .env, which suggests it was rotated, but revocation is unverified.

Evidence

```
+  TELEGRAM_BOT_TOKEN: 8872***
+  KUBESIGHT_API_TOKEN: ksa_***
```

Impact

The ksa_ token carries its owner's full permissions (tokens have no scopes), so it can call every REST and MCP tool that user can. With the bot token, an attacker can read and drain the approval bot's updates (blocking approvals), message approvers as the bot, and repoint its webhook.

Recommendation

- Revoke the ksa_ token, and revoke the bot token with BotFather `/revoke`.

- Purge the file from history on all branches with `git filter-repo --path k8s/hermes-secret.yaml --invert-paths`, then force-push both remotes.
- Add gitleaks or trufflehog to pre-commit and CI.

HIGH

PLAT-05 The nodes/proxy get ClusterRole allows exec in every pod on every node, contrary to the manifest's comment

Location k8s/node-stats-rbac.yaml:23-35

Confidence Likely. This is publicly documented kubelet behaviour (January 2026); it was not reproduced.

Who can exploit Anyone who obtains the backend identity: a backend RCE (the backend runs as root, PLAT-12), a leaked service-account token, or the static token copy in /root/.kube/config. The file tells operators to bind this role on every cluster.

Description

The kubelet maps the HTTP method to the RBAC verb, and WebSocket `/exec`, `/attach` and `/portforward` requests arrive as GET. So `get` on `nodes/proxy` is enough to run commands in any container on the node, including privileged system pods.

Evidence

```
#   GET path on each node's kubelet (pod lists, logs, metrics). No create, so no
#   exec or attach.
resources: ["nodes/proxy"]
verbs: ["get"]
```

Impact

Node and cluster takeover on every cluster where this role is applied.

Recommendation

Get the Summary API data another way: metrics-server, a metrics DaemonSet, or a narrowly scoped dedicated identity. At minimum, correct the comment, mark the role as equivalent to cluster-admin, and keep it opt-in.

MEDIUM

PLAT-06 A local branch holds Hermes runtime secrets (LLM OAuth refresh tokens, Telegram, ksa_)

Location branch `backup-before-clean-push`, commits 26342ac0, 383b582b and 51962ee8. Files: hermes/hermes/.hermes/.env, hermes/.hermes/.env.txt, hermes/hermes/.hermes/auth.json, .bash_history, hermes-local.runtime.env, kanban.db.

Confidence Confirmed

Who can exploit Anyone who gets a copy of .git (a backup, a copied worktree, or a `push --all` / `--mirror`). No remote head contains these commits today (checked with ls-remote).

Description

The branch holds:

- OpenAI-Codex id, access and refresh tokens;
- Nous Portal access and refresh tokens and an agent key;
- another Telegram bot token and another ksa_ token (`ksa_0518***`);
- shell history containing tokens.

The hermes/ folder is now gitignored, but the same files remain in the working tree.

Evidence

```
26342ac0:hermes/hermes/.hermes/.env  TELEGRAM_BOT_TOKEN=8872***  KUBESIGHT_API_TOKEN=ksa_***
383b582b:auth.json  openai-codex refresh_token rt.1***  nous refresh_token rt_B***
```

Impact

Persistent access to the LLM accounts, plus the same bot and KubeSight access as PLAT-02.

Recommendation

- Delete the branch, then run `git reflog expire --expire=now --all` and `git gc --prune=now`.
- Revoke the Codex and Nous sessions, and rotate the bot and ksa_ tokens.
- Keep backups of the Hermes home outside the repository tree.

MEDIUM

PLAT-08 Version caps in requirements.txt block security fixes (gunicorn, flask-cors, cryptography, pytest)

Location backend/requirements.txt (lines 2, 11, 12, 13); requirements.txt:2

Confidence Likely. The version ranges are verified; the advisory data comes from the auditor's knowledge, not pip-audit.

Who can exploit A remote client, for request smuggling when the front proxy parses Transfer-Encoding differently from gunicorn; anyone using flask-cors path-matching flaws to bypass the CORS policy.

Description

- `gunicorn<23` keeps CVE-2024-6827 (request smuggling, fixed in 23.0.0).
- `flask-cors<5` keeps CVE-2024-6844, CVE-2024-6866 and CVE-2024-6839 (fixed in 6.0.0).
- `cryptography<44` keeps CVE-2024-12797 (fixed in 44.0.1) and CVE-2026-26007 (fixed in 46.0.5).
- `pytest<9` ships in the runtime image.

Evidence

```
flask-cors>=4.0.0,<5.0.0
cryptography>=42.0.0,<44.0.0
gunicorn>=21.2.0,<23.0.0
pytest>=8.0.0,<9.0.0
```

Impact

Request desync in front of per-route permission checks, plus vulnerable crypto and test tooling in the runtime image.

Recommendation

- Require `gunicorn>=23`, `flask-cors>=6` and `cryptography>=46.0.5`.
- Move pytest to a dev-only requirements file.
- Build a hashed lockfile with `pip-compile --generate-hashes`, and run pip-audit in Bitbucket CI.

MEDIUM

PLAT-11 The session JWT lives in localStorage and there is no CSP, so any XSS becomes full takeover

Location frontend/src/authStorage.js:4-8; frontend/nginx.conf:17-19; frontend/index.html:10; backend/api/frontend_static.py:18-35

Confidence Confirmed

Who can exploit Anyone who manages to inject script: a dependency compromise, a rendering bug (APR-02 is one today, on a server-rendered page), or a malicious browser extension.

Description

The 8-hour bearer token with full RBAC sits in localStorage. There is no CSP and no HSTS, and /api responses carry no security headers. When Flask serves the app in single-container mode, it sets no security headers at all. No XSS sink was found in the React app itself: there is no dangerouslySetInnerHTML, and React 19 blocks `javascript:` hrefs.

Evidence

```
localStorage.setItem(TOKEN_KEY, token);
```

Impact

A single XSS steals a token that can apply YAML or Helm, reveal secrets and mint new tokens.

Recommendation

- Add a strict CSP in nginx and Flask: `script-src 'self'` plus a hash for the theme snippet, `object-src 'none'`, `base-uri 'none'`, `frame-ancestors 'self'`.
- Add HSTS.
- Remove the `new Function` loader (PLAT-21).
- Longer term, move the session to an HttpOnly, SameSite=Strict cookie with a CSRF token.

MEDIUM

PLAT-12 The backend runs as root and keeps a static copy of the service-account token

Location backend/Dockerfile (no USER); backend/k8s_entrypoint.sh:16-37; frontend/Dockerfile

Confidence Confirmed

Who can exploit Anyone who gets code execution in the backend, for example through YAML, Helm, Excel or archive parsing, or the root-run kubectl, helm, tofu and git.

Description

gunicorn runs as root and holds KUBESIGHT_SECRET_KEY, the decrypted kubeconfigs, and the vSphere and SSH credentials. The entrypoint copies the projected token into /root/.kube/config once, so the copy stops following token rotation. The backend Deployment and RBAC manifests were removed from the repo in bb8a2519, so its securityContext, automountServiceAccountToken and the deployer ClusterRole (described in DOCUMENTATION.md:300 as "full CRUD on ... secrets") cannot be reviewed.

Evidence

```
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
kubectl config set-credentials kubesight-sa --kubeconfig=/root/.kube/config --token="${TOKEN}"
```

Impact

Any RCE runs as root inside the container, with access to every credential KubeSight manages.

Recommendation

- Add a non-root USER, and use `tokenFile:` in the kubeconfig so token rotation is honoured.
- Commit a hardened backend Deployment: runAsNonRoot, readOnlyRootFilesystem, drop ALL capabilities, RuntimeDefault seccomp.
- Use nginx-unprivileged for the frontend.

MEDIUM

PLAT-13 The admin control plane is on the external ingress class with no source restriction

Location	k8s/ingress.yaml:9
Confidence	Likely. That the class is internet-facing is inferred from its name.
Who can exploit	Anyone on the network the external controller serves, possibly the internet.

Description

The UI, all of /api (including /api/mcp) and /health go through `ingress-nginx-external`. There is no source allowlist, no auth gateway and no rate limit.

Evidence

```
kubernetes.io/ingress.class: ingress-nginx-external
```

Impact

A management plane equivalent to cluster-admin is reachable from outside, which compounds AUTH-01 and AUTH-05.

Recommendation

- Use an internal ingress class or VPN, or add a source allowlist.
- Expose only the webhook paths externally, through a separate Ingress.
- Add a request-rate limit on /api/auth/login.

MEDIUM

PLAT-14 The Werkzeug debugger PIN is committed, and the dev server binds 0.0.0.0 with debug on by default

Location	backend/server.stderr.log:7 (tracked); backend/app.py:15-16
Confidence	Likely. The PIN stays valid while the machine, user and app path are unchanged.
Who can exploit	Anyone on the same LAN or VPN as a developer running `python app.py`.

Description

The tracked log shows "Debugger is active!" on a private 172.31.x.x address with the PIN in clear. FLASK_DEBUG defaults to true outside production.

Evidence

```
 * Running on all addresses (0.0.0.0)
 * Debugger PIN: 141-***
```

Impact

A Python console, which means RCE, on a developer workstation that holds kubeconfigs and Hermes credentials.

Recommendation

Bind the dev server to 127.0.0.1 and make debug opt-in. Run `git rm --cached backend/server.*.log` and ignore `*.log`.

LOW PLAT-16 Bitbucket PR pipelines run untrusted code, and repository secrets are probably reachable from them

Location bitbucket-pipelines.yml:54, :134-138

Confidence Likely. Variable scoping is set in the Bitbucket UI and could not be checked.

Who can exploit Anyone who can push a branch and open a PR.

Description

PR pipelines run `pip install`, `npm ci` and the tests, all of which the PR controls. Bitbucket exposes repository variables (DOCKER_PASSWORD, BITBUCKET_API_TOKEN) to same-repo PR pipelines unless they are deployment-scoped. Images are not digest-pinned, and node:20 has been end of life since 2026-04-30.

Evidence

```
- echo "$DOCKER_PASSWORD" | docker login ...
pull-requests: {"**": [parallel: [backend-tests, frontend-build]]}
```

Impact

The registry push credential can be stolen.

Recommendation

Move these variables into a deployment environment used only by the image and release steps. Pin images by digest, move to Node 22, and disable caches on PR pipelines.

LOW PLAT-17 Outdated and unverified tool binaries in the backend image

Location backend/Dockerfile (HELM_VERSION=v3.17.2, KUBECTL_VERSION=v1.32.2); Dockerfile.ci-imagetools; k8s/ci-buildkitd.yaml

Confidence Likely

Who can exploit For the Helm issues, users with chart import or deploy rights who upload crafted charts.

Description

- Helm 3.17.2 has denial-of-service CVEs on crafted charts: CVE-2025-32386 and CVE-2025-32387 (fixed in 3.17.3), and CVE-2025-55198 and CVE-2025-55199 (fixed in 3.18.5).
- kubectl 1.32 is past end of life.
- kubectl and helm are downloaded with no checksum check.
- Base images are not digest-pinned, and the vSphere provider is cloned from a mutable git tag.

Evidence

```
&& curl -fsSL "https://get.helm.sh/helm-${HELM_VERSION}-linux-amd64.tar.gz" | tar xz -C /tmp
```

Impact

A crafted chart can exhaust or crash the backend, and the image build is exposed to supply-chain tampering.

Recommendation

Move to Helm 3.18.5+ and kubectl 1.34+. Verify both downloads against their SHA-256 sums. Pin images by digest and the provider by commit SHA.

LOW PLAT-18 .gitignore gaps, and log files are tracked

Location .gitignore; backend/.gitignore; frontend/.gitignore

Confidence Confirmed

Who can exploit No one directly; this raises the odds of the next accidental commit.

Description

`git check-ignore` shows these are not ignored: `*.log` (the server logs are tracked), root `.env`, `frontend/.env`, `*.pem`, `*.key` and a bare `kubeconfig`. `*.db` is covered in backend/ but not at the repository root.

Impact

More accidental commits like PLAT-02, MCP-01 and PLAT-06.

Recommendation

- Ignore `*.log`, `.env*` (keeping `!.env.example`), `*.pem`, `*.key`, `*.p12`, `*.p8`, `*kubeconfig*`, `*.db`, `*.tfstate*` and `*.runtime.env`.

- Untrack the log files.
- Add a gitleaks hook.

LOW PLAT-19 Weak dev credentials in the compose files, published on all interfaces

Location docker-compose.yml:12-14; docker-compose.test.yml:12; docker-compose.hermes-local.yml

Confidence Confirmed

Who can exploit Anyone on the developer's LAN.

Description

Postgres publishes 5432 and 5433 on all interfaces with `kubesight/kubesight` and `kubesight_test` credentials. The Hermes local API_SERVER_KEY is a hard-coded literal, but it is bound to localhost.

Evidence

```
POSTGRES_PASSWORD: kubesight
ports: ["5432:5432"]
```

Impact

The dev database can be read and written from the LAN.

Recommendation

Bind to 127.0.0.1 and read the passwords from an ignored .env file.

INFO PLAT-20 hermes-mcp-egress.yaml targets the wrong namespace

Location k8s/hermes-mcp-egress.yaml:20

Confidence Confirmed

Who can exploit No one.

Description

The policy lives in `kubesight`, but the shipped Hermes deploys to `kubesight-hermes`, so as shipped it matches nothing.

Impact

Operators may "fix" connectivity by loosening the base Hermes policy instead.

Recommendation

Ship the file with `namespace: kubesight-hermes`.

INFO PLAT-21 The runtime config.js loader never runs in production, and uses new Function when it does

Location frontend/src/runtimeConfig.js:14-21

Confidence Confirmed

Who can exploit No one.

Description

With `base: "./"`, `new URL("config.js", "./")` always throws, so PUBLIC_API_URL is silently ignored. If the code were fixed as written, it would evaluate the fetched text, which would force `unsafe-eval` into any CSP.

Recommendation

Load it with a `<script src="config.js">` tag or fetch JSON instead, and build the URL from `document.baseURI`.

INFO PLAT-22 A component heartbeat token is placed in the URL query string

Location frontend/src/pages/ComponentsPage.jsx:65; backend/api/routes/topology_components.py:82

Confidence Confirmed

Who can exploit Anyone who can read access logs.

Description

`?token=<webhookToken>` ends up in ingress logs and browser history. The token only authorizes heartbeats.

Impact

Forged heartbeats could hide health alerts (see also INT-07).

Recommendation

Accept the token in a header instead.

7. Dependency scan

Frontend results come from `npm audit --package-lock-only` on 2026-10-11. Every frontend advisory is in a build or test-time package; the runtime bundle contains only react and react-dom 19.2.7. Python advisories come from the auditor's knowledge as of mid-2026, because pip-audit was not available.

Package	Installed / allowed	Advisory	Severity	Fixed in
gunicorn	22.0.0 / >=21.2.0,<23.0.0	CVE-2024-6827 (request smuggling)	High	23.0.0 (blocked by cap)
flask-cors	4.0.2 / >=4.0.0,<5.0.0	CVE-2024-6844, -6866, -6839	Medium	6.0.0 (blocked by cap)
cryptography	43.0.3 / >=42.0.0,<44.0.0	CVE-2024-12797, CVE-2026-26007	Low–Medium	44.0.1 / 46.0.5 (blocked by cap)
pytest (shipped at runtime)	8.4.2 / <9	CVE-2025-71176	Low	9.0.3 (blocked by cap)
PyJWT	2.13.0 / >=2.8.0	CVE-2026-32597	Medium	2.12.0 (installed version fine; floor too low)
Flask	3.1.3 / >=3.0.0	CVE-2026-27205	Low	3.1.3 (installed version fine)
Werkzeug (transitive)	3.1.8	CVE-2025-66221 / CVE-2026-21860	Low	3.1.4 / 3.1.5 (installed version fine)
paramiko, PyYAML, psycopg2-binary, openpyxl, pyotp, segno, pyvmomi, fpdf2, defusedxml, python-dotenv, Flask-SQLAlchemy	ranges	None known	–	–
All Python dependencies	ranges, no hashes, no lockfile	Builds are not reproducible	Low	pip-compile --generate-hashes
helm (binary)	v3.17.2	CVE-2025-32386/32387/55198/55199 (DoS)	Medium	3.17.3 / 3.18.5
kubectl (binary)	v1.32.2	End of life	Low	1.34+
tinypool (via vitest)	<=2.1.1	GHSA-5gmw-xhrv-c9v3, GHSA-85c8-ppgw-ccpr	Critical (dev only)	vitest 5.0.3
vitest / @vitest/mocker	3.2.6	GHSA-82fw-gwwq-j7x9	Moderate (dev only)	4.1.11+
browserslist	<=4.28.6	GHSA-c83g-rgw3-j3cx, GHSA-73wf-gq98-2v4g	High (build only)	npm audit fix
nanoid	3.3.12	GHSA-28wg-ghj8-5hjv, GHSA-2v37-7h3g-55p8	High (build only)	3.3.18
postcss	8.5.15	GHSA-r28c-9q8g-f849, GHSA-fxqj-rqcc-2cmp	High (build only)	8.5.23
source-map-js	<=1.2.1	GHSA-68fv-2mgg-jv7q	High (build only)	1.2.2
baseline-browser-mapping	<2.11.0	GHSA-w5vr-8v7q-w6rv	Moderate (build only)	2.11.0
react / react-dom	19.2.7	None (CVE-2025-55182 affects react-server-dom-*, not used)	–	–
vite	6.4.3	None reported (dev server binds 0.0.0.0)	–	–
root and backend package-lock.json	empty files	–	–	Delete them

8. What is already done well

Authentication, sessions and RBAC

- **Sessions.**
  - JWTs are pinned to HS256, and a purpose claim separates full sessions from onboarding, MFA and download tokens.
  - A version claim lets logout, password reset, disable, lock and MFA reset kill existing sessions on the server.
  - Lifetimes are short: 8 h sessions, 30 min interim tokens, 2 min download tickets.
- **Production boot.** It refuses to start with a missing or default signing key, a missing or default encryption key, or authentication disabled. The debug server is off, and the k8s entrypoint runs gunicorn.
- **First-login gate.** It applies to API tokens too.
- **API tokens.** They are 160-bit random, stored only as a sha256 hash, shown once, and only an admin can mint one for someone else.
- **Passwords.**
  - Hashed with PBKDF2.
  - Temporary passwords are random, single-use and expiring.
  - A 12-character complexity policy applies at the first change.
  - In production, a temporary password is not shown on screen when the email fails.
- **MFA.** TOTP seeds are encrypted at rest, and MFA failures are counted separately.
- **Account safeguards.** The last admin cannot be disabled, locked or deleted, and users cannot disable, lock or delete themselves.
- **Hermes service account.** Its boundary is enforced in three places and re-applied on every boot.
- **Machine-to-machine endpoints.** All use per-object secrets compared in constant time, and the ticketing inbound check refuses everything when no secret is set.
- **Errors and audit.** Forbidden attempts are audited, a generic 500 never leaks a traceback, and /health carries no installation data.

Kubernetes cluster access and mutation

- **Names checked before kubectl.** Kubernetes names are validated as DNS-1123 before any argv is built, and every kubectl and helm argv is screened again for connection and identity flags (backend/api/k8s_provider.py:177-188, backend/api/services/helm_service.py:159-164).
- **Safe subprocess calls.** Commands use argument lists with no shell and have bounded timeouts. Exec commands sit after `--` and are length-capped.
- **Safe YAML parsing.** Only `yaml.safe_load` and `safe_load_all` are used in the reviewed paths. No template engine renders user input there.
- **Kubeconfigs encrypted at rest.** Per-call plaintext copies are 0600 in a 0700 directory (tmpfs when available) and are deleted after use.
- **Secret YAML redaction.** Without `secrets:reveal`, Secret YAML is redacted, including the last-applied annotation. Only the redacted form is cached, and a parse failure returns nothing rather than leaking. Applying YAML that still contains the redaction marker is refused.
- **Per-pod log checks.** Log access is checked per pod and per container (backend/api/services/logs_service.py:194, 288).
- **Safe chart import.** Archive import refuses links, skips special files and caps the total size.
- **One approval gate for writes.** YAML, Helm, restart, scale and rollback share one gate that fails closed and refuses calls that have neither a user nor an approval context.
- **Default deny.** Users with no access rules and no legacy grants get nothing.

Native CI platform

- **Inbound webhook verification.** Comparisons are constant-time everywhere (HMAC-SHA256 or the raw secret). A trigger or service with no stored secret rejects every delivery. Secrets are 32 random bytes, stored encrypted, with audited reveal and rotate.

- **Generic webhooks cannot widen what the owner allowed.**
  - A ref is accepted only with the override switch or a mapping, and inputs only from `allowed_inputs`; asking for more is refused outright.
  - Refs and commits are grammar-checked.
  - Each trigger has a rate limit and a cap on refs per delivery.
  - Builds run as the last person who saved the trigger, and that person's rights are re-checked on every delivery.
- **Machine credentials.** Worker callback tokens are random per build, stored hashed, compared with `compare_digest`, and refused once the build ends (with a short grace window for artifacts only). Agent tokens and task claim tokens are stored only as hashes.
- **Build pod hardening.**
  - Containers run as uid 65532 with a read-only root filesystem, no privilege escalation, all capabilities dropped and RuntimeDefault seccomp.
  - Pods get no ServiceAccount token and `enableServiceLinks: false`.
  - The namespace enforces restricted PSA, and each build has a NetworkPolicy that blocks ingress.
  - Git authentication goes through `GIT_CONFIG_*` environment variables, never argv or the URL.
  - Registry credentials are mounted only into the image stage.
- **Secret handling.** Secrets are injected per stage, masked in logs on ingest, and never included in serializers.
- **File handling.** Artifact filenames are sanitized and stored under a confined root, XML is parsed with defusedxml, and the workspace browser lists names only, with validated and quoted paths.
- **Approval stages.** Decisions are POST-only and limited to named approvers or `ci_builds:approve` holders. Self-approval is off by default, a person cannot vote twice, and votes are row-locked.
- **Deploy targets.** Saving a target requires `apps:deploy` and namespace access. The stamped user's rights are re-checked at run time, and the deploy goes through `apply_yaml` with the approval, promotion and registry gates.
- **CI Assist.** Proposals are re-validated on accept and saved through the normal pipeline path, and supplying secret values requires `ci_secrets:manage`.

Integrations, webhooks and outbound requests

- **Secrets at rest.** Stored third-party secrets are Fernet-encrypted (Zoho, Jira, Jenkins, registry, SMTP, Bitbucket, Play, App Store, Telegram, vSphere). GET responses return only "configured" booleans, and saving a blank field keeps the stored value instead of sending it back to the browser.
- **Inbound ticketing webhooks.** They fail closed when no secret is configured and compare the secret in constant time.
- **Bitbucket access.** Repository URLs are allowlisted to `https://bitbucket.org/<ws>/<repo>` with no userinfo, query or fragment. Metadata calls refuse redirects and validate every API URL against api.bitbucket.org and the expected repository prefix.
- **Git credentials.** They are supplied through `GIT_ASKPASS` and environment config, never in URLs or argv, and git stderr is not reflected.
- **Analysis and PR job hardening.** Jobs run non-root with a read-only root filesystem, all capabilities dropped, no privilege escalation, seccomp RuntimeDefault and no service-account token. Each job gets its own NetworkPolicy, and in local-Docker mode the build container has no network.
- **Worker callbacks.** They use per-analysis random tokens stored as hashes, and artifact downloads check path containment and SHA-256.
- **Mobile.**
  - Upload and download filenames are sanitized, and store paths are id-based.
  - Play and App Store API hosts are hardcoded, with path parameters quoted.
  - Re-signing keystores never leave Jenkins, and IPA plist parsing rejects entity declarations.
- **Jenkins client.** It does not follow redirects, and it rebases URLs returned by Jenkins onto the configured base URL.
- **Email.** Alert, deployment-request and change-bundle emails escape their HTML, and ticket comments are posted as plain text.

- **Deploy automation.** Kubernetes names and variable names are validated before reaching kubectl, agent and MCP starts go through the ticket validator, and cluster approval gates apply at handoff.
- **Hermes endpoint.** The endpoint and token come only from the environment and cannot be set by users.

MCP server and Hermes agent

- **Permission and access checks on every call.** Every tool re-checks its permission at call time. Cluster tools go through `resolve_cluster` (`can_access_cluster` + `check_cluster`) and `require_namespace`, and reuse the same services as the UI. All 34 mutating tools are flagged `write=True`; none is mislabelled as read.
- **Per-cluster policy coverage.** The policy is applied to inventory ids, change bundles, promotions and ticket deploy targets, and cross-cluster listings are filtered with `keep_allowed`.
- **No self-approval through MCP.** Approving deployment requests, change bundles, ticket actions or VM requests is never exposed as a tool. VM requests always need a separate VM approver: the requesting account cannot approve, and Telegram needs a non-empty `vm_approvers` list.
- **Ticket-agent writes are validated on the server, not just in the prompt.**
  - Targets must match the catalog exactly.
  - Tags and variable names are regex-checked.
  - Env-var changes only touch existing literal variables.
  - Agent and MCP runs are re-validated in `start_run`.
- **Telegram.** The chat id is checked, and an @channel with no approver list fails closed. Callbacks carry a per-task nonce, claims are atomic, and messages are plain text.
- **Tokens and errors.** API tokens are hashed, shown once, and checked for expiry and inactive users. Auth is header-only, so there is no CSRF on /api/mcp, and unexpected exceptions return a generic message.
- **Audit.** Every MCP call is audited (without its arguments), and policy changes are audited.
- **Hermes runtime.**
  - hermes-local.runtime.env is sanitized and gitignored, and the local compose binds to 127.0.0.1 only.
  - The production Hermes manifest runs non-root with a read-only root filesystem, drops all capabilities, mounts no service-account token and limits egress.
  - The Application Intelligence Hermes runs without MCP.
- **hermes-agent account.** It has a fixed minimal role and no interactive login, and it cannot receive cluster access rules.

Cluster Builder, SSH and VM provisioning

- **No credentials in the OpenTofu config.** tofu_config.render emits only object ids and UUIDs. vCenter credentials (VSPHERE_*) and the state-backend token (TF_HTTP_*) are passed only through the process environment, so the downloadable main.tf.json holds no credentials. The config is built from a fresh vCenter read and never trusted from the browser.
- **No shell or HCL injection found.**
  - The cluster name is regex-validated, and vCenter ids must match `^[A-Za-z0-9_.:-]{1,80}$`.
  - main.tf.json is written with `json.dump`, so values are JSON-escaped.
  - Node names and addresses are shlex-quoted.
  - Disk path, VIP interface, registry prefixes, repo and proxy URLs, and NFS server, export and mount options are all regex-validated and quoted.
- **OpenTofu HTTP state backend locked down.**
  - Requests that do not come from loopback are refused. The check fails closed, and there is no ProxyFix or X-Forwarded-For trust, so loopback cannot be spoofed.
  - Each job also uses basic auth with its own `token_urlsafe(32)` token. Only its SHA-256 is stored, it is valid only while the job is active, and it is compared with `hmac.compare_digest`.
  - State is Fernet-encrypted at rest, with proper lock, serial and lineage handling.
- **Two-person destroy.** The requester cannot approve their own destroy. What gets applied is the saved binary plan, so the approved plan is the applied plan. Recovery applies automatically only when every resource matches the originally approved addresses.

- **VM-request separation.** Hermes can only ask. Nothing is created until a VM approver approves the real plan, and VM approvers are a separate list from deploy approvers and from `cluster_builds:execute`. Self-approval is blocked, and Telegram presses fail closed when there is no approver list.
- **Secrets never serialized.** SSH credentials, vSphere passwords, the provisioning password, VRRP auth, the kubeadm certificate key and the join command are kept in Fernet-encrypted columns. The API only reports whether each one is configured. Logs are scrubbed, and tofu output also redacts the vCenter password and the state token.
- **Host-key hygiene.** Stale tofu keys for reused addresses are deleted before connecting, and a changed recorded key is always refused.
- **Safe day-two growth.** The etcd snapshot taken before control-plane growth is written 0600 in a 0700 directory. Growth plans that would change, replace or delete a running VM are blocked.
- **The leftover test file is clean.** backend/tests/test_cluster_provisioning_ssh_real.py uses only a 127.0.0.1 paramiko test server, a generated RSA key and a dummy password. It holds no real hosts, kubeconfigs or production credentials. It is safe to keep and worth committing on purpose.

Approvals, promotions and governed delivery

- **One vote per voter.** Each voter can vote once per request or bundle, enforced by DB unique constraints, and emails are lowercased consistently.
- **No self-approval.** Self-approval is blocked by account id and by email for deployment requests, bundles, CI stages, Cluster Builder destroys and VM requests.
- **Bundles frozen after submit.** Items cannot be edited once a bundle leaves draft. The executor applies the stored `yaml_preview` and re-checks registry and ladder rules at execution.
- **CI Approval stage.** Approvers, quorum and self-approval settings are snapshotted at stage start, and decisions take a row lock.
- **Hermes approvals.** They are claimed atomically with a status compare-and-set. Telegram presses check the chat, the nonce and the approver lists, and VM approval applies only the exact planned job id.
- **Approval tokens are not session tokens.** They carry a `typ` claim and no `sub`, so they cannot be used to log in.
- **Promotion releases.** They check namespace access for each target, and exception bundles always require at least one approval.
- **Approvals hub.** It is read-only and filters each kind by that kind's own permission; votes go to each kind's own route.
- **Excel import.** XML parsing is protected by defusedxml.

Frontend, infrastructure, secrets and dependencies

- **No frontend XSS sinks.** Nothing in frontend/src uses dangerouslySetInnerHTML, dynamic innerHTML, document.write, eval, postMessage or iframes. Ticket, Hermes, YAML, log and alert text all render as React text, React 19.2.7 blocks `javascript:` hrefs, and downloads use `download=` with non-HTML MIME types.
- **Routing and bundle.** Hash routing drops undeclared query keys, which blocks `?next=` open redirects. The bundle contains no secrets; the only VITE_* variable is the API base URL.
- **CI, analysis and Hermes namespaces.** They enforce the restricted Pod Security level, mount no service-account tokens, use namespaced roles with no PV or PVC delete, set quotas and limit ranges, and give each build its own NetworkPolicy. Hermes egress is public port 443 only, with RFC1918 excluded.
- **BuildKit.** It runs rootless and unprivileged as UID 1000 with no host mounts, and the manifest documents its trade-offs.
- **Image builds.** Tofu, Trivy, Syft and Hadolint downloads are checksum-verified. The worker and imagetools images run as non-root, the production Hermes image is digest-pinned and non-root, and corporate CAs are passed as BuildKit secrets.
- **Production startup.** The entrypoint defaults to production with debug off, and production enforcement refuses dev keys and AUTH_REQUIRED=false.
- **Secrets handling.** The areeba-txm chart templates no secret values. The Hermes home, runtime env, backend/data, databases and instance/ are now ignored, and the private keys in tests are dummies.
- **Self-CI.** PR pipelines only run tests; image push happens on tags or manual runs.

9. Remediation roadmap

Once the priority actions are done, the remaining Medium and Low findings group into a few pieces of work. Each group can ship on its own.

Workstream	Findings	Shape of the fix
One outbound-URL validator	K8S-08, INT-02, INT-04, CI-10, PROV-05	A shared helper: https only, resolve the host and refuse loopback, link-local, metadata and the cluster API; no redirects. Clear stored secrets when a URL changes.
Approval model clean-up	APR-04, APR-05, APR-06, APR-07, APR-12, APR-13, K8S-06, K8S-11, PROV-04	Votes keyed by user id from a snapshotted pool, POST-only single-use email links, a maximum window, every rule change and vote audited, fail closed when the requester is unknown.
Session and login hardening	AUTH-05, AUTH-06, AUTH-07, AUTH-10, AUTH-13, AUTH-14, AUTH-15, PLAT-11	Throttle instead of hard-lock, refuse locked users on tokens, single-use onboarding and TOTP steps, scoped and expiring API tokens, CSP and HSTS, then move the session to an HttpOnly cookie.
CI hardening, second pass	CI-05 to CI-14	BuildKit per trust group with mTLS and real egress rules, collector confinement, per-secret scopes, merge-check pipeline restrictions, signed webhook timestamps.
Secret exposure in responses	INT-03, INT-07, PROV-01, APR-14, MCP-12	Return only what each permission needs: no webhook URLs to viewers, no heartbeat tokens to viewers, `secrets:reveal` for workload copy.
Platform and supply chain	PLAT-08, PLAT-12, PLAT-13, PLAT-14, PLAT-16 to PLAT-19	Non-root images, an internal ingress, a hashed lockfile with pip-audit in CI, digest-pinned images, gitleaks, tighter .gitignore.
Hermes and agents, second pass	MCP-08 to MCP-11, INT-06, INT-12	Agent-side container enforcement, batch caps, numeric Telegram ids mapped to users, HMAC-signed ticket webhooks keyed by provider.